

Camada de Transporte

Camadas de Transporte e Aplicação

Agenda

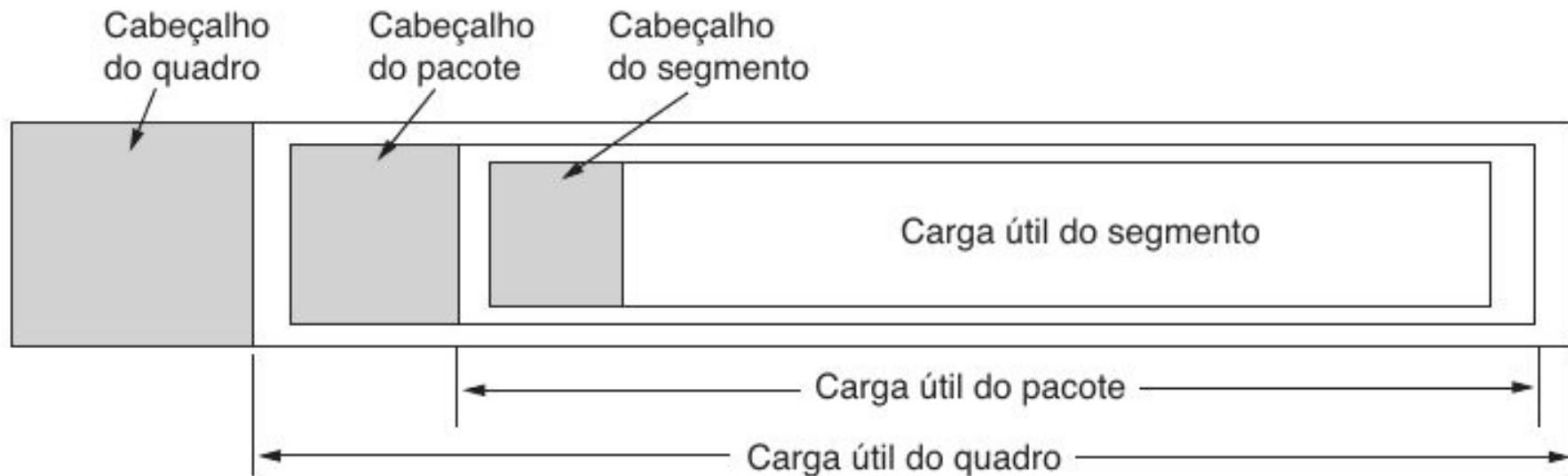
- Introdução
- Primitivas para um Serviço de Transporte Simples
- *User Datagram Protocol* (UDP)
- *Transport Control Protocol* (TCP)

Introdução

Principais Protocolos de Transporte

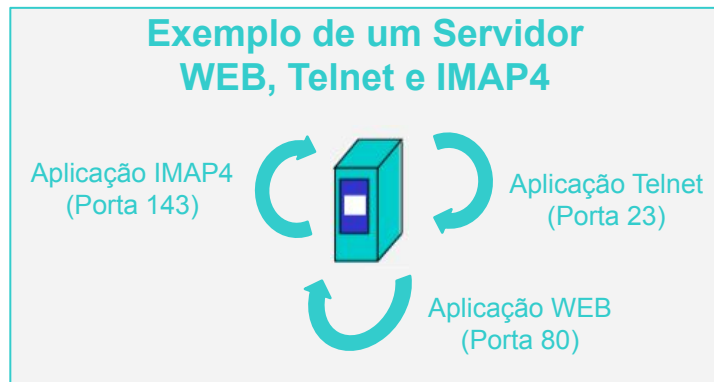
- *User Datagram Protocol (UDP)*
 - Sem conexões
 - Transferência não confiável de dados
- *Transport Control Protocol (TCP)*
 - Orientado à conexão
 - Transferência confiável de dados
 - Controle de fluxo
 - Controle de congestionamento

Segmento: Mensagem na Camada de Transporte

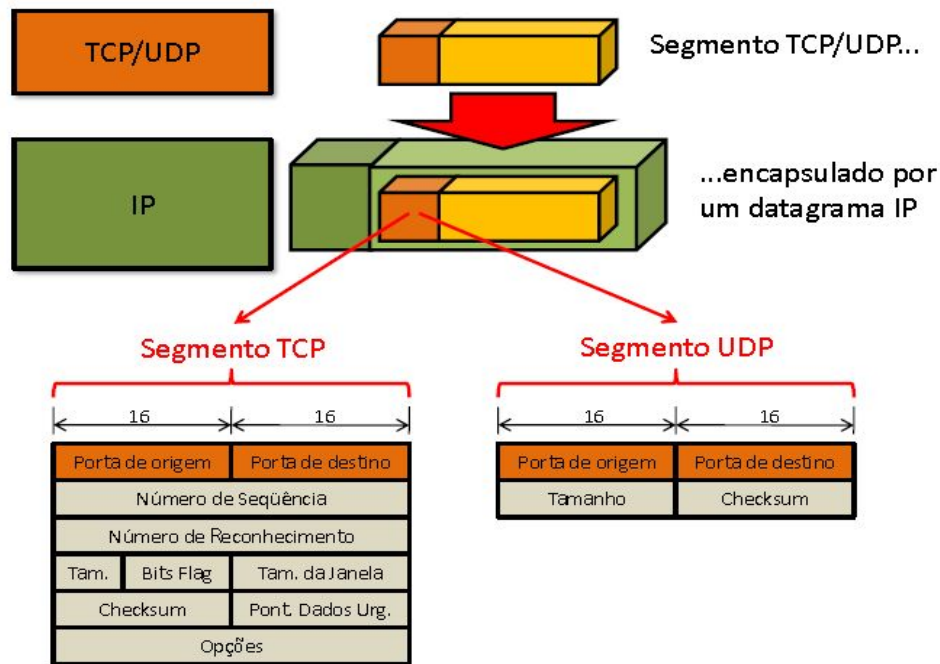


Porta

- Número (16 bits) que identifica um processo de transporte em uma máquina
- A porta é o endereço de transporte (e.g, na camada de rede da Internet, o número IP é o endereço de cada máquina na rede)



Formato do Cabeçalho no Segmento TCP/UDP



Portas Classificadas em Três Faixas *

- Portas bem conhecidas (0 a 1023) são reservadas para serviços padrão, como HTTP (porta 80) e SMTP (porta 25)
- Portas registradas (1024 a 49.151) são atribuídas a aplicações específicas que foram registradas oficialmente
- Portas dinâmicas ou privadas (49.152 a 65.535) são geralmente usadas de forma temporária por clientes que estabelecem conexões com servidores

* Conforme definido pela Internet Assigned Numbers Authority (IANA)

Portas Bem Conhecidas

- Do inglês, *well-known ports*
- Reservadas para protocolos de aplicação tradicionais (entre 0 e 1023)

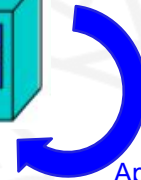
Porta	Descrição
20/TCP	FTP - porta de dados
21/TCP	FTP - porta de controle
22/TCP,UDP	SSH
23/TCP,UDP	Telnet
25/TCP,UDP	SMTP
53/TCP,UDP	DNS
80/TCP	HTTP
81/TCP	HTTP Alternativa
110/TCP	POP3
143/TCP,UDP	IMAP4
194/TCP	IRC
366/TCP,UDP	SMTP
989/TCP,UDP	FTP - porta de dados sobre TLS/SSL
990/TCP,UDP	FTP - porta de controle sobre TLS/SSL
993/TCP	IMAP4 sobre SSL
995/TCP	POP3 sobre SSL

Exemplo de Aplicação Telnet (Porta 23)

host A

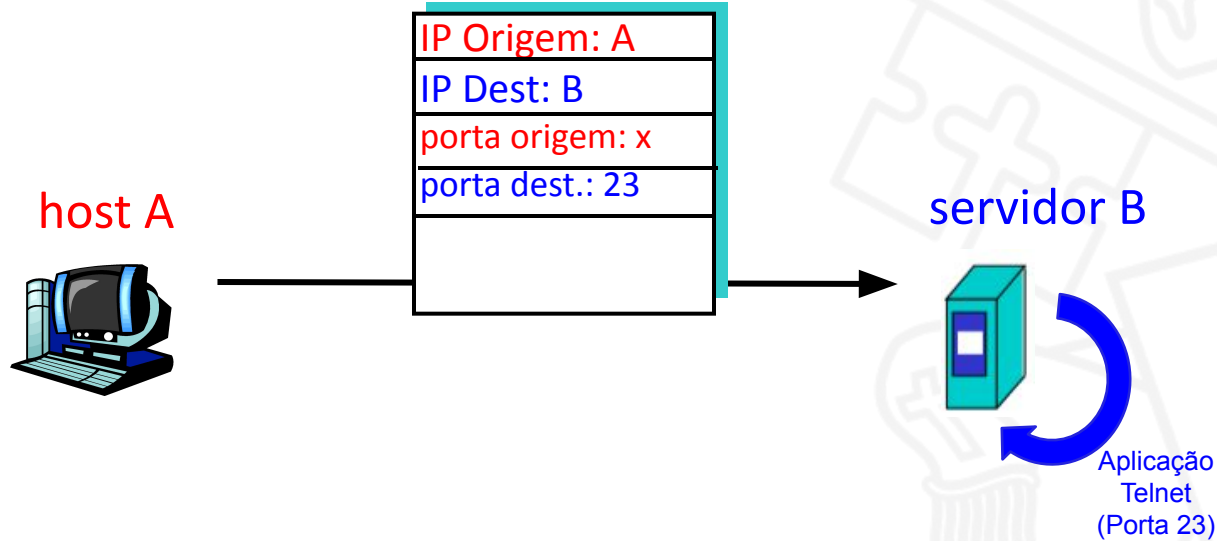


servidor B

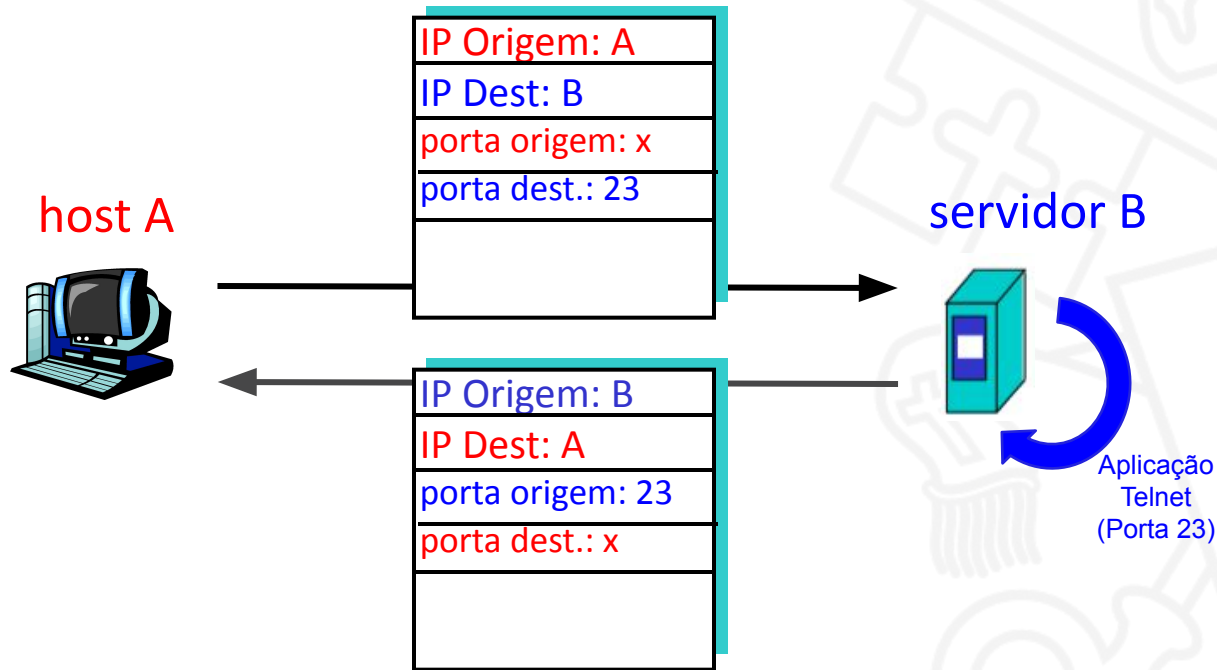


Aplicação
Telnet
(Porta 23)

Exemplo de Aplicação Telnet (Porta 23)



Exemplo de Aplicação Telnet (Porta 23)

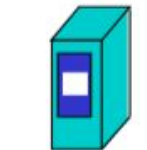


Exemplo de Aplicação WEB (Porta 80)

Cliente WEB
host A



servidor
WEB B

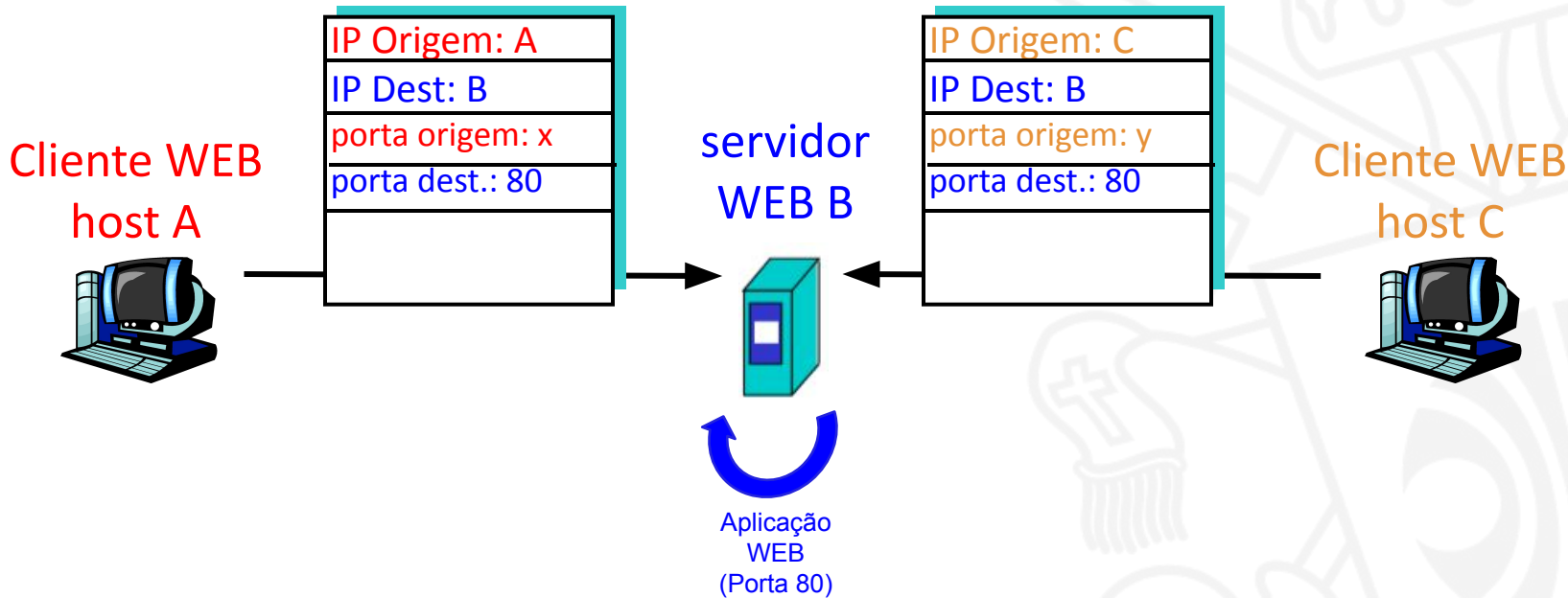


Aplicação
WEB
(Porta 80)

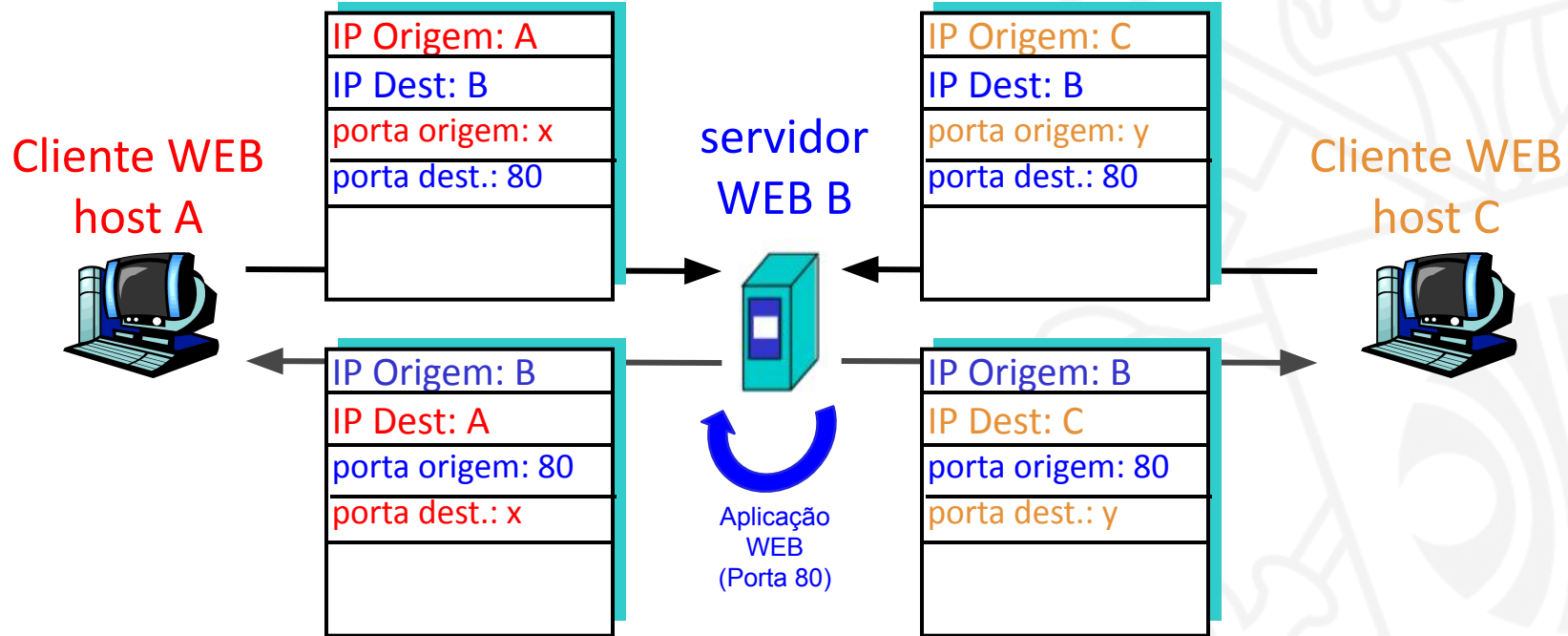
Cliente WEB
host C



Exemplo de Aplicação WEB (Porta 80)



Exemplo de Aplicação WEB (Porta 80)



Camada de Rede vs. de Transporte

- Responsáveis pelo transporte de dados entre origem e destino
- Contudo, têm preocupações distintas:
 - **Camada de rede**: roteamento dos dados
 - **Camada de transporte**: comunicação entre **processos** que executam na origem e destino. Tais processos executam os protocolos de transporte

Unificar as Camadas de Rede e Transporte?

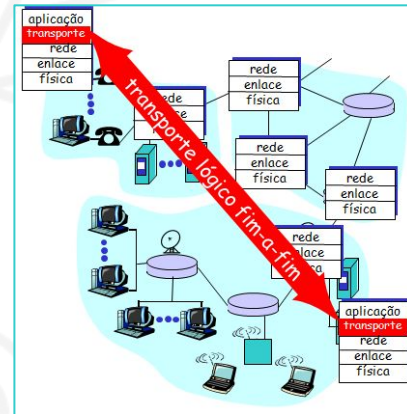
- Se as camadas de rede e transporte oferecem serviços semelhante, as duas não deveriam ser unificadas?
 - Resposta sutil, mas crucial
 - Os protocolos de transporte funcionam integralmente nas máquinas de origem e destino e os de rede, principalmente, nos roteadores

Camada de Rede vs. de Transporte

- Camada de rede
 - Executa o transporte fim a fim usando datagramas ou circuitos virtuais
- Camada de transporte
 - Utiliza a camada de rede
 - Provê as abstrações necessárias para que a aplicação use rede

Comunicação Lógica Fim a Fim

- Significa que uma entidade no nó emissor se comunica diretamente com sua entidade-par do *host* destinatário
- A camada de transporte é a primeira a fazer comunicação lógica fim a fim
- Roteadores, *hubs* e *switches* não precisam analisar os cabeçalhos de transporte para executar suas funções nativas



Camada de Rede vs. de Transporte

Camada de Rede	Camada de Transporte
Transferência de dados entre sistemas finais	Transferência de dados entre processos
Funciona principalmente nos roteadores	Funciona inteiramente nas máquinas dos usuários
Identificam as máquinas nas redes (e.g, número IP)	Identificam os processos nas máquinas (e.g., porta)

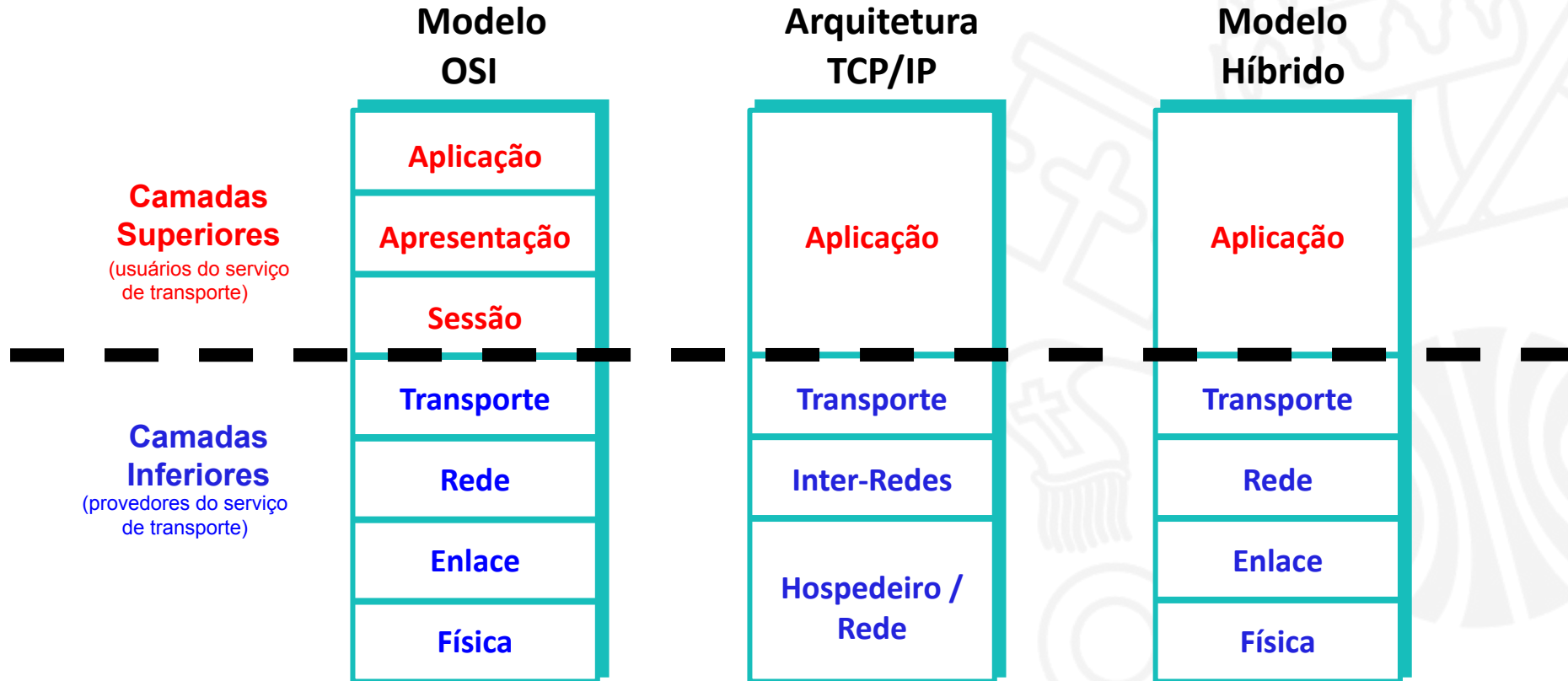
Serviços Oferecidos pela Camada de Transporte

- Normalmente são processos pertencentes à camada de aplicação
- Devem ser confiáveis, eficientes e econômicos
- Devem abstrair os problemas da rede para a aplicação
 - Problemas acontecem: rede não perfeita, heterogênea e dinâmica
 - Usuários não possuem qualquer controle real sobre a camada de rede

Serviços Orientados ou não à Conexão

	Rede orientada à conexão	Rede não orientada à conexão
Camada de Transporte orientado à conexão		Exemplo: TCP/IP
Camada de Transporte não orientado à conexão	Combinação normalmente inexistente porque estabelecería uma conexão para enviar um único pacote	Exemplo: UDP/IP

Distinção entre Camadas: Superiores e Inferiores

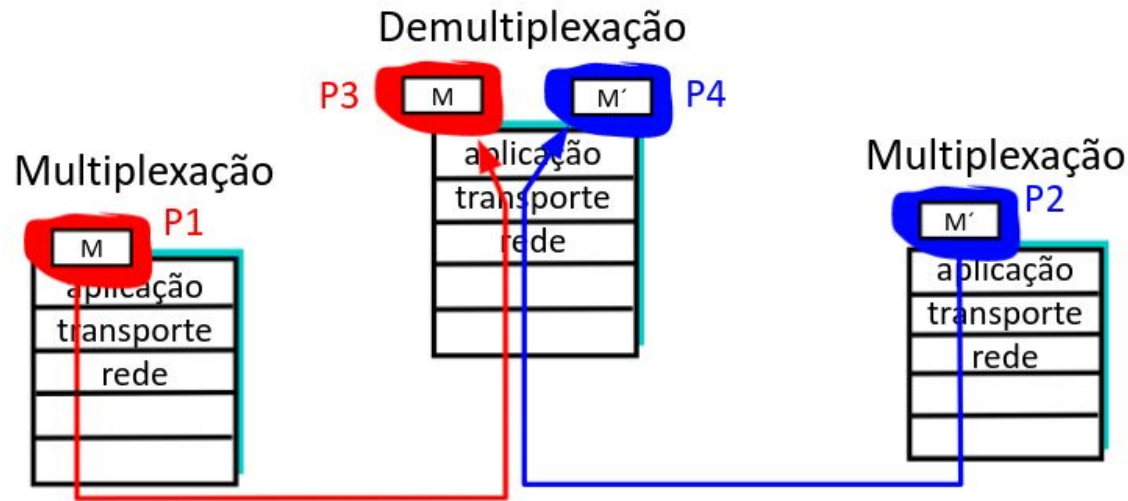


Função da Camada de Transporte

- Dar sentido à pilha de protocolos:
 - Tornando as camadas superiores imunes à tecnologia, projeto e imperfeições da rede
 - Efetuando retransmissões ou restabelecendo conexões

Multiplexação e Demultiplexação de Aplicações

- Permite a comunicação entre processos executando em máquinas distintas



Multiplexação e Demultiplexação de Aplicações

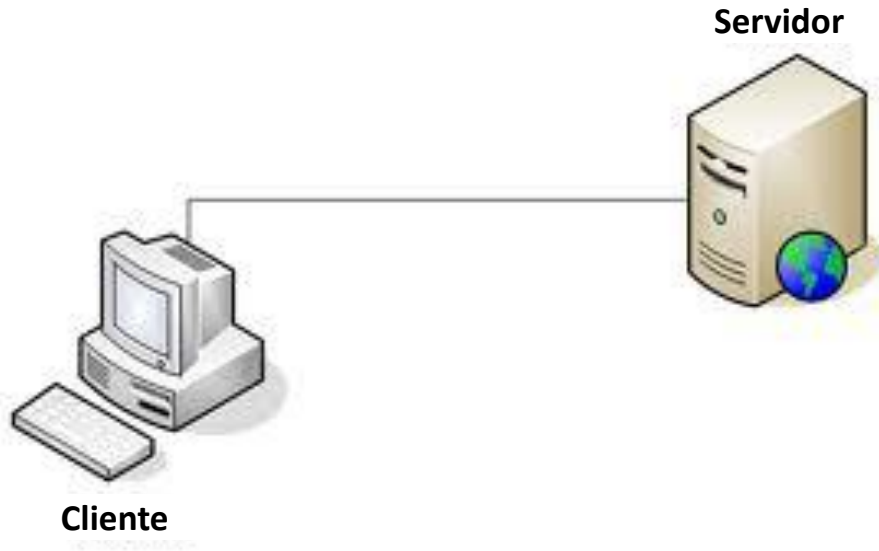
- **Multiplexação** (origem): Reunir os dados provenientes de diferentes processos de aplicação
- **Demultiplexação** (destino): Entrega dos dados contidos em um segmento da camada de transporte ao processo de aplicação correto

Primitivas para um Serviço de Transporte Simples

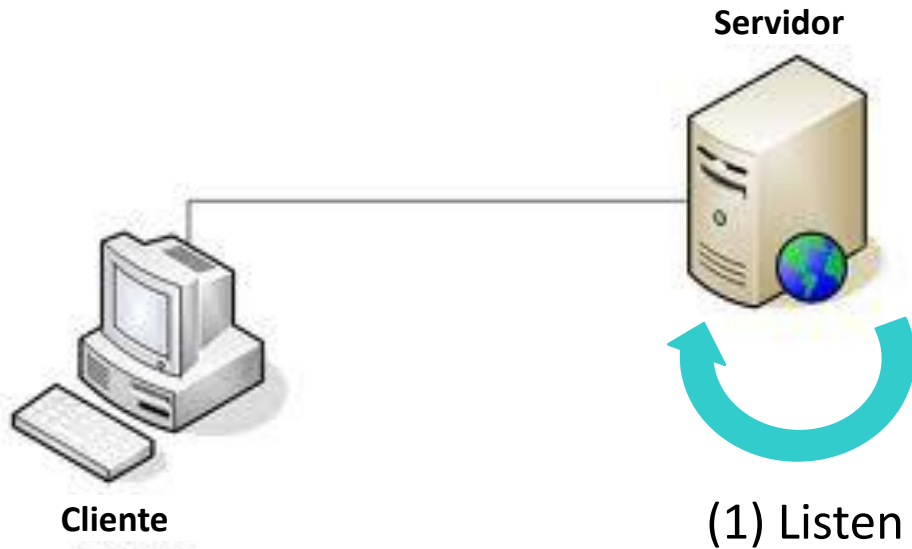
Primitivas

Primitiva	Pacote enviado	Significado
LISTEN	(nenhum)	Bloqueia até algum processo tentar conectar
CONNECT	CONNECTION REQ.	Tenta ativamente estabelecer uma conexão
SEND	DATA	Envia informação
RECEIVE	(nenhum)	Bloqueia até que um pacote de dados chegue
DISCONNECT	DISCONNECTION REQ.	Solicita uma liberação da conexão

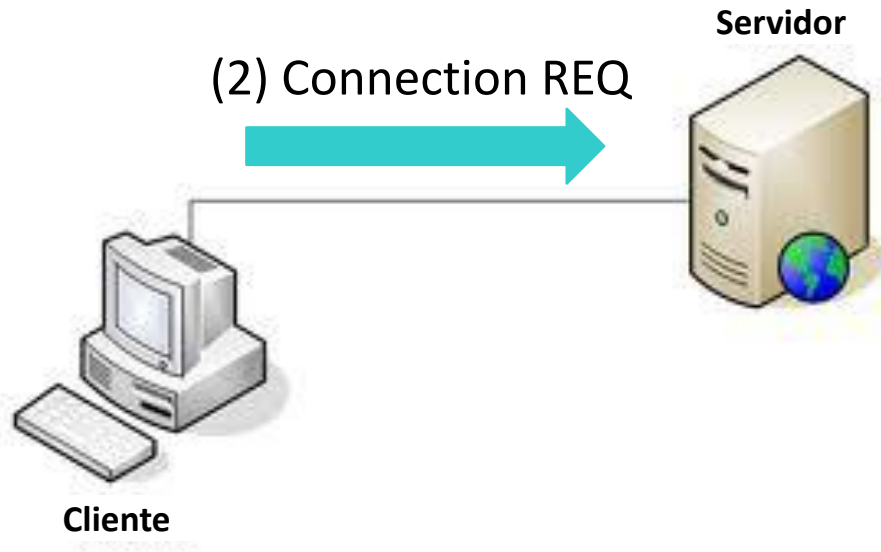
Primitivas



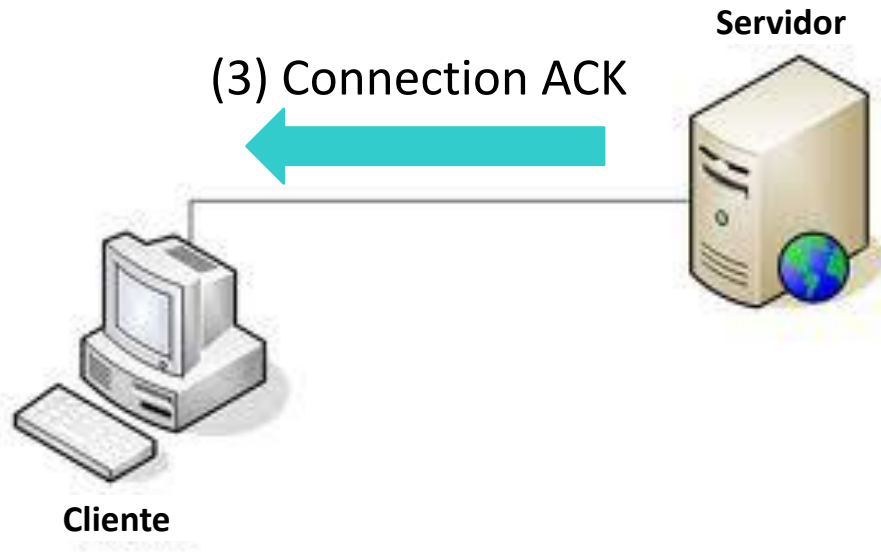
Primitivas



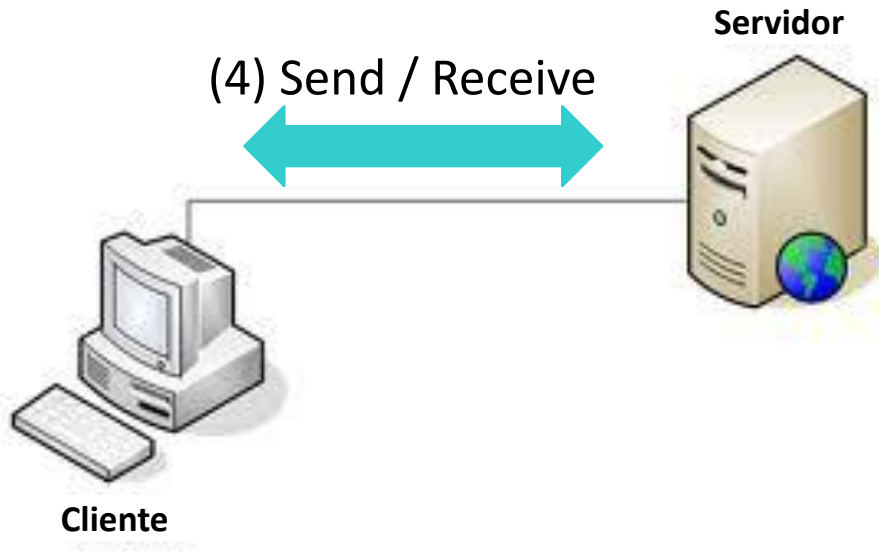
Primitivas



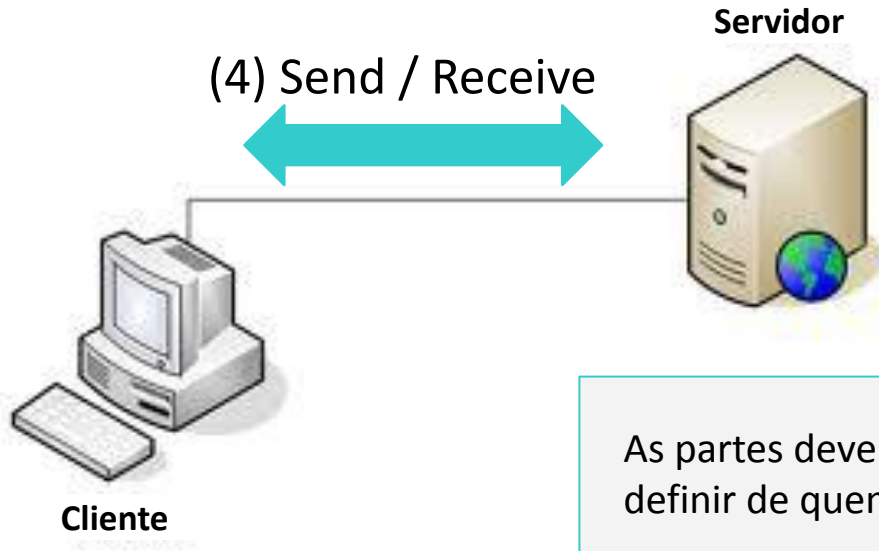
Primitivas



Primitivas

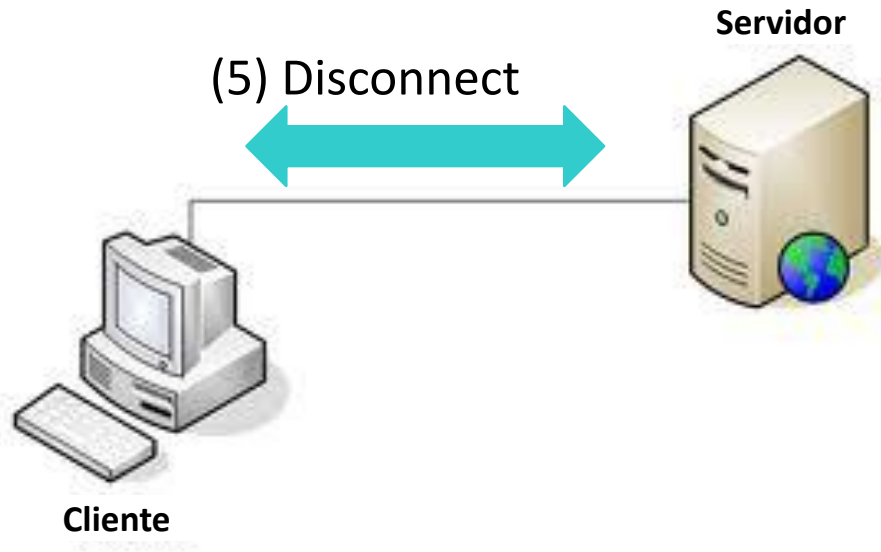


Primitivas

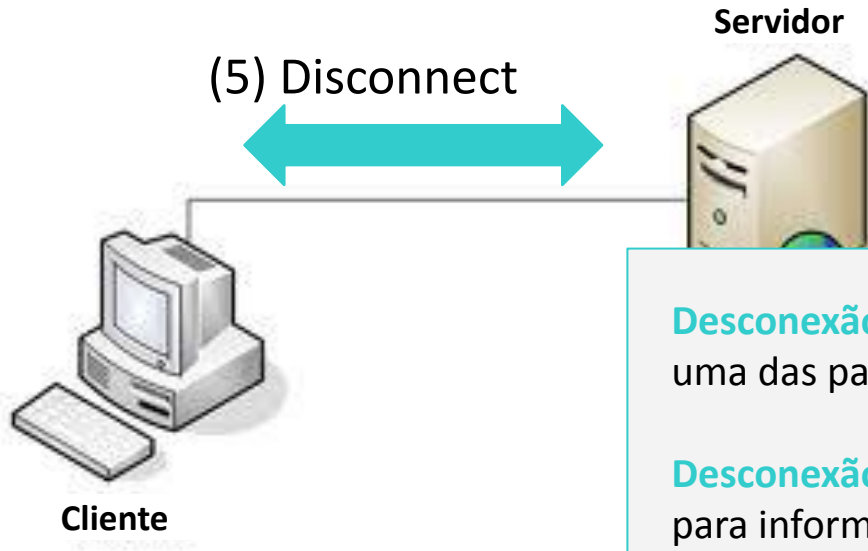


As partes devem ter uma política para definir de quem é a vez de enviar dados

Primitivas

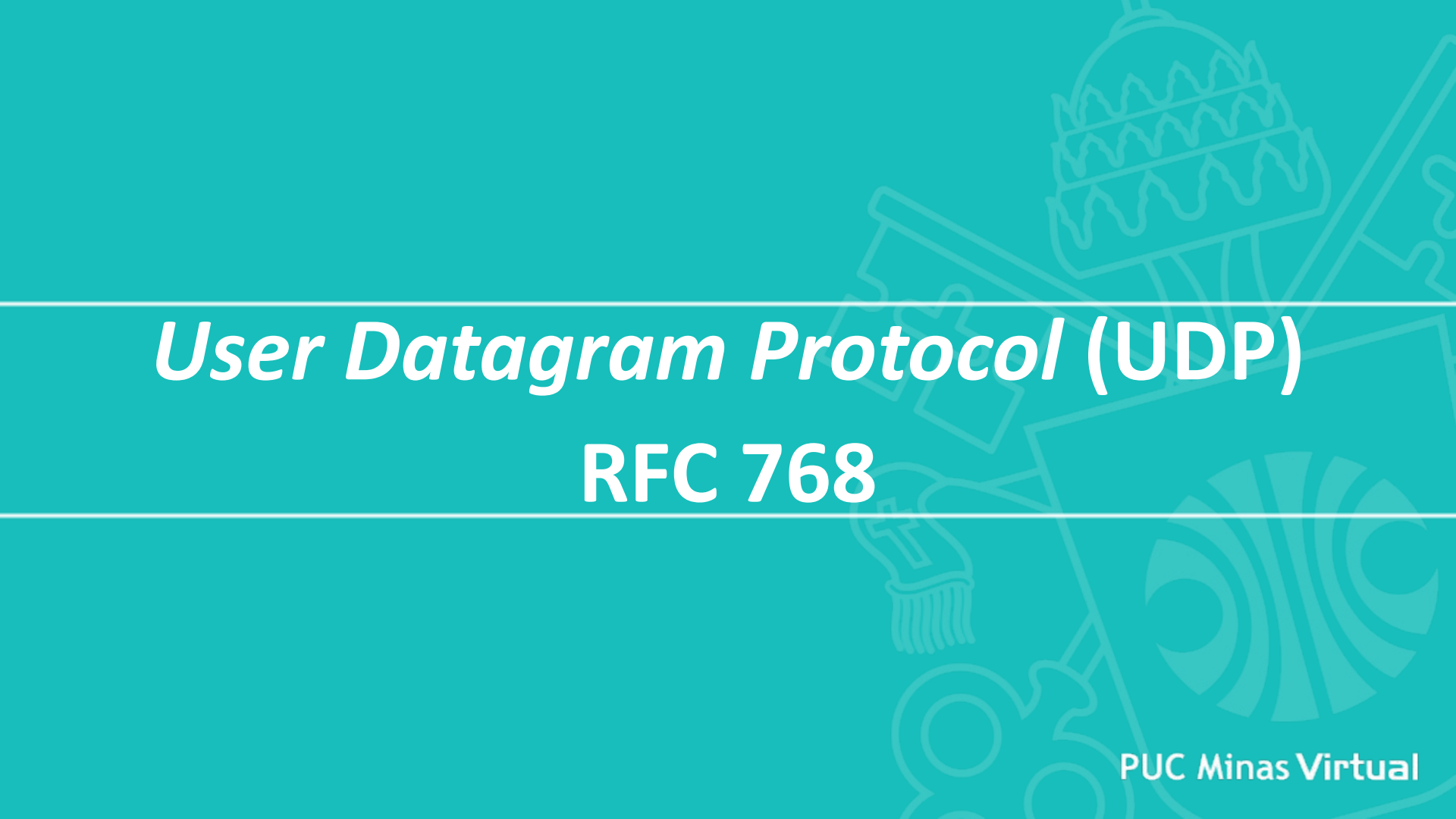


Primitivas



Desconexão assimétrica: a conexão termina quando uma das partes solicita

Desconexão simétrica: cada parte faz sua desconexão para informar que não enviará mais dados



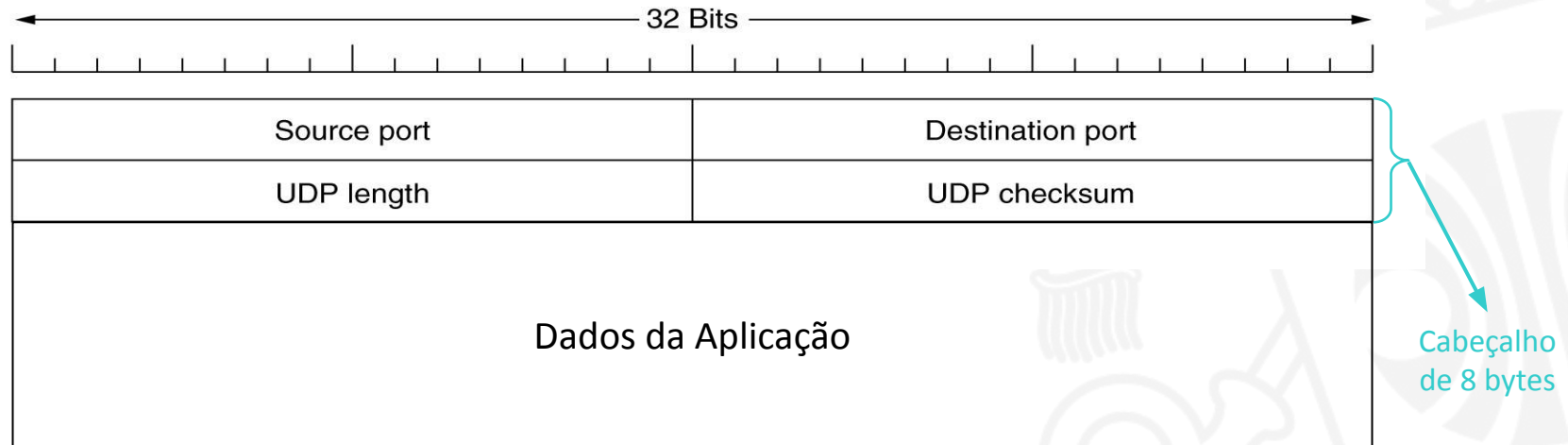
User Datagram Protocol (UDP)

RFC 768

UDP

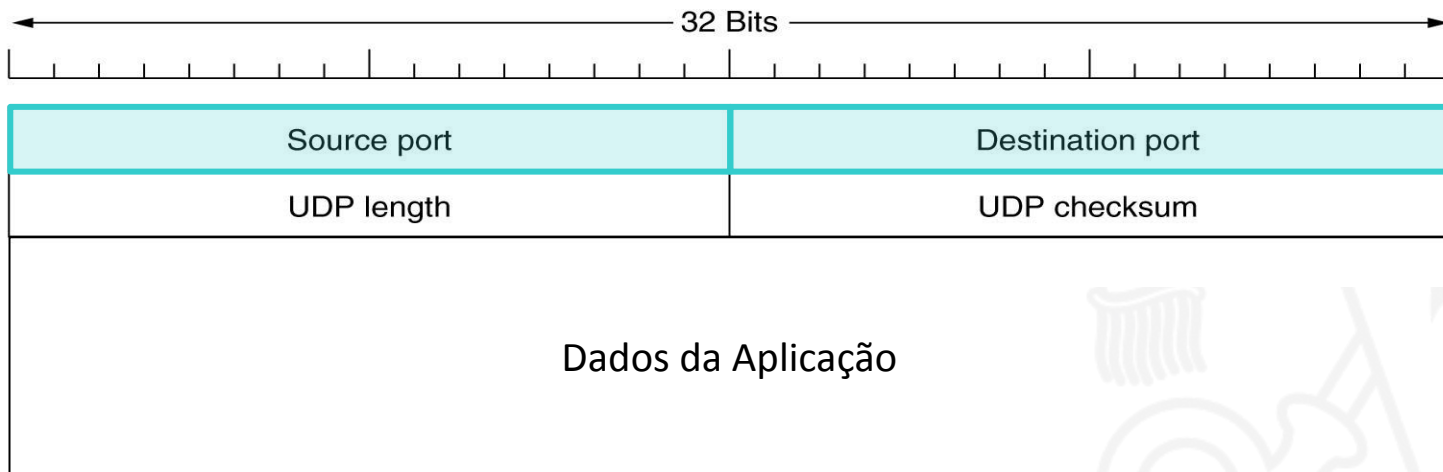
- Protocolo de transporte não confiável
- Não estabelece conexão
 - Não há apresentação entre o UDP emissor e o receptor
 - Cada segmento UDP é tratado de forma independente dos outros
- Faz um serviço *best-effort*, sendo que seus segmentos podem ser:
 - Perdidos
 - Entregues fora de ordem para a aplicação

Segmento UDP



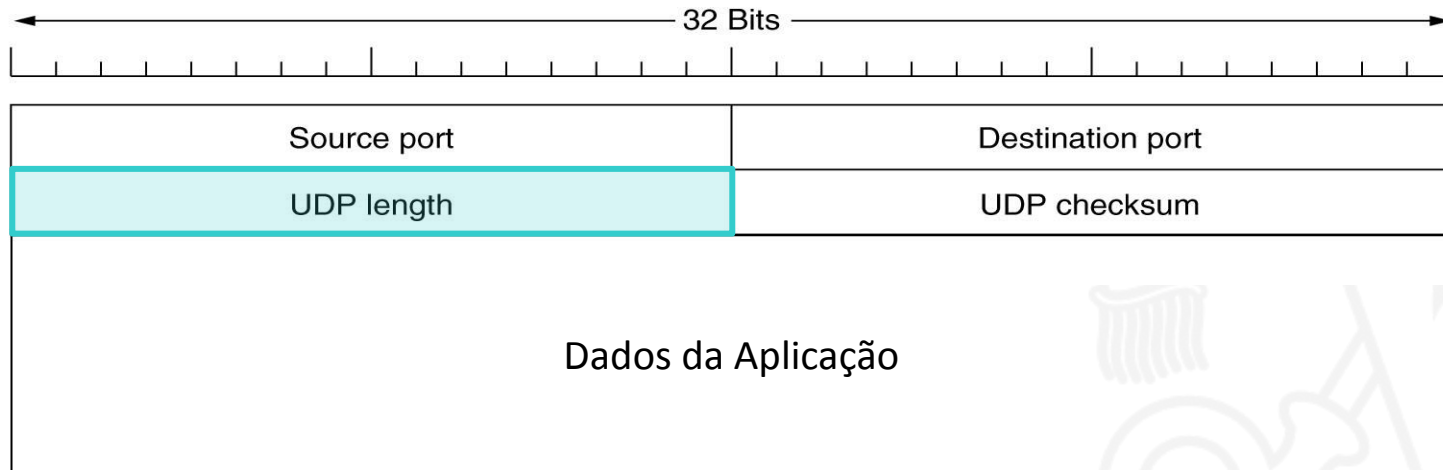
Source Port e Destination Port (16 bits, cada)

- Identificam os processos na origem e destino. Quando um segmento UDP chega, sua carga útil é entregue ao processo associado à porta destino



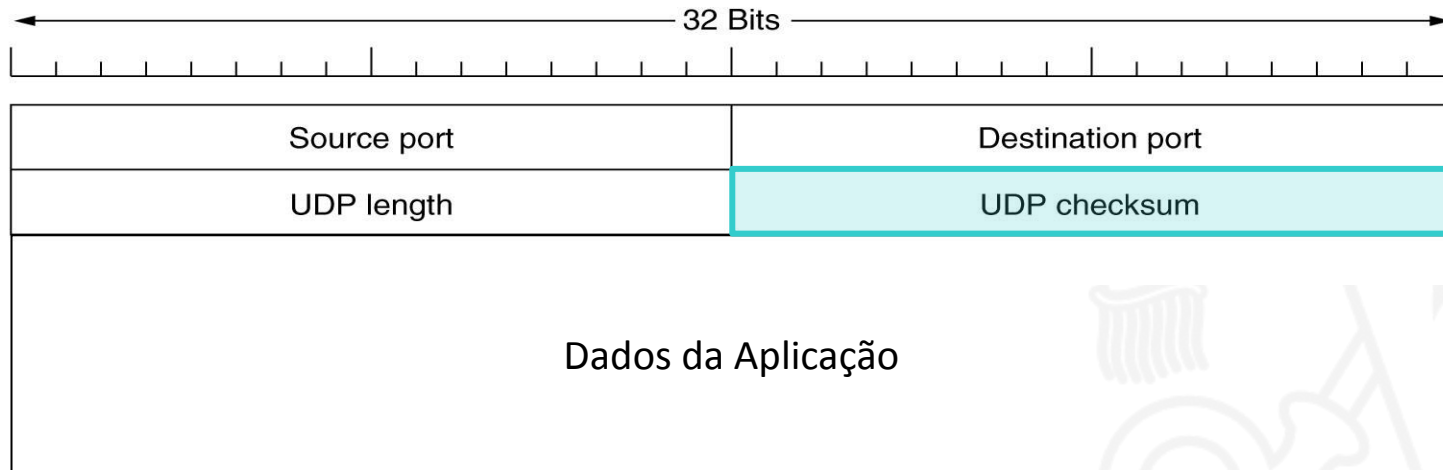
UDP length (16 bits)

- Tamanho total do segmento (incluindo o cabeçalho) em bytes (entre 8 de 65515)



UDP checksum (16 bits)

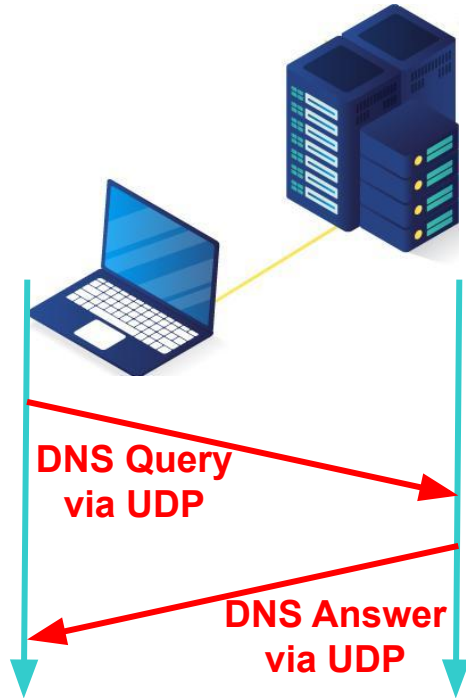
- Campo opcional, armazenado como 0 se não for calculado



Funcionamento Básico do UDP

- Servidor inicializado, rodando e aguardando um segmento
- Cliente envia uma solicitação curta para o servidor
- Cliente aguarda por uma resposta curta
- Servidor recebe o pedido e devolve a resposta
- Quando o cliente não recebe a resposta, ele aguarda um *timeout* e **pode** tentar novamente

DNS: Um Exemplo de Aplicação UDP



Ação do UDP

- Fornece uma interface para o protocolo de rede com o recurso adicional de multiplexação / demultiplexação de vários processos que utilizam as portas

Ações que o UDP Não Realiza

- Controle de fluxo
- Controle de congestionamento
- Controle de erros
- Retransmissão após a recepção de um segmento incorreto

Por que UDP?

- Não há estabelecimento de conexão e, portanto, não introduz atrasos
- Não há estado de conexão no emissor nem no receptor
- Pequeno *overhead* no cabeçalho do segmento
- Taxa de envio não regulada (não há controle de congestionamento)

Onde Usamos UDP: Aplicações *One Shot*

- Aplicações cliente-servidor com apenas uma requisição e uma resposta
- O custo para estabelecer uma conexão é alto quando comparado com a transferência de dado
- Aplicações de multimídia contínua (*streaming*)
 - Tolerantes à perda
 - Sensíveis à taxa

Exemplo de Aplicações UDP e TCP

Aplicação	Protocolo de camada de aplicação	Protocolo de transporte
Correio eletrônico	SMTP	TCP
Acesso a terminal remoto	Telnet	TCP
Web	HTTP	TCP
Transferência de arquivo	FTP	TCP
Servidor remoto de arquivo	NFS	UDP
Recepção de multimídia	proprietário	UDP
Telefonia por Internet	proprietário	UDP
Gerenciamento de rede	SNMP	UDP
Protocolo de roteamento	RIP	UDP
Tradução de nome	DNS	UDP



Transport Control Protocol (TCP)

RFCs: 793, 1122, 1323, 2018, 2581

Agenda

- Introdução
- Primitivas para um Serviço de Transporte Simples
- *User Datagram Protocol (UDP)*
- ***Transport Control Protocol (TCP)***

- Introdução
- Primitivas de Soquetes para TCP
- Cabeçalho do Segmento TCP
- Serviços do TCP

Agenda

- Introdução
- Primitivas para um Serviço de Transporte Simples
- *User Datagram Protocol (UDP)*
- ***Transport Control Protocol (TCP)***

- **Introdução**
- Primitivas de Soquetes para TCP
- Cabeçalho do Segmento TCP
- Serviços do TCP

TCP

- Fornece um fluxo de bytes fim a fim confiável em uma rede nem sempre confiável
- Capaz de se adaptar dinamicamente às condições da rede tornando-a robusta às falhas
- Principal protocolo de transporte na arquitetura TCP/IP

Etapas do Modelo de Serviço do TCP

- 1) Entidade TCP emissora recebe o fluxo de dados da aplicação e o divide em segmentos
- 2) Entidade TCP emissora envia cada segmento para a camada de rede e essa efetua sua tarefa
- 3) Ao receber os segmentos, a entidade TCP receptora restaura o fluxo de dados original

Características do TCP

- **Transferência confiável de dados:** garante que os dados serão entregues da forma em que foram enviados
- **Orientado à conexão:** conexões gerenciadas nos sistemas finais
- **Controle de fluxo:** emissor não esgota a capacidade do receptor
- **Controle de congestionamento:** emissor não esgota os recursos da rede
- **Gerenciamento de temporizadores:** baseado em temporizadores

Soquetes (do inglês, *Sockets*)

- *Application Programming Interface* (API) que permite a comunicação entre processos
- Permite que uma aplicação se comunique com outra sem se preocupar com detalhes da pilha
- Baseados no padrão Berkeley sockets
- Identificação do soquete = endereço IP + número de porta

Conexão entre Soquetes

- Acontece quando o soquete da origem se conecta ao do destino
- É necessário que a origem conheça o IP e porta do destino
- Um soquete pode ser utilizado por várias conexões ao mesmo tempo (duas ou mais conexões podem terminar no mesmo soquete)

Agenda

- Introdução
- Primitivas para um Serviço de Transporte Simples
- *User Datagram Protocol (UDP)*
- *Transport Control Protocol (TCP)*

- Introdução
- **Primitivas de Soquetes para TCP**
- Cabeçalho do Segmento TCP
- Serviços do TCP

Primitivas de Soquetes para TCP

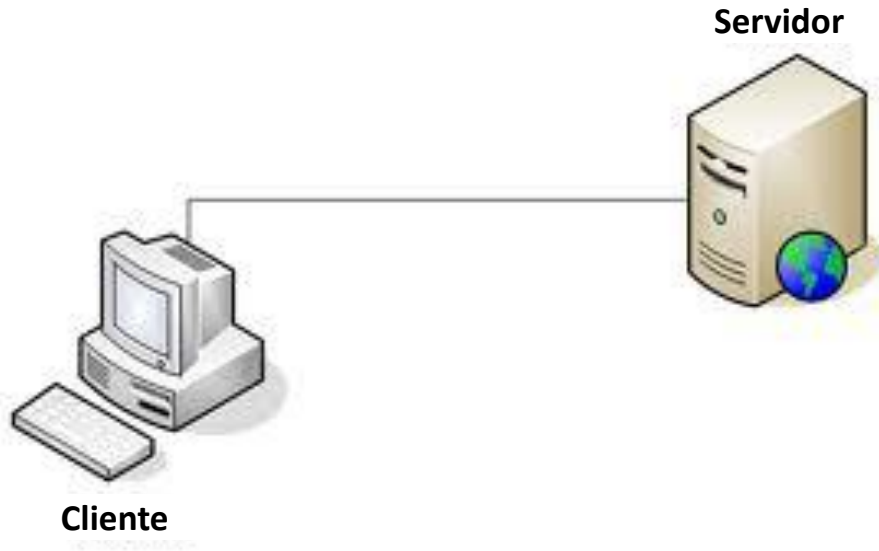
Primitiva	Significado
SOCKET	Criar um novo ponto final de comunicação
BIND	Anexar um endereço local a um soquete
LISTEN	Anunciar a disposição para aceitar conexões; mostrar o tamanho da fila
ACCEPT	Bloquear o responsável pela chamada até uma tentativa de conexão ser recebida
CONNECT	Tentar estabelecer uma conexão ativamente
SEND	Enviar alguns dados através da conexão
RECEIVE	Receber alguns dados da conexão
CLOSE	Encerrar a conexão

Primitivas de Soquetes para TCP

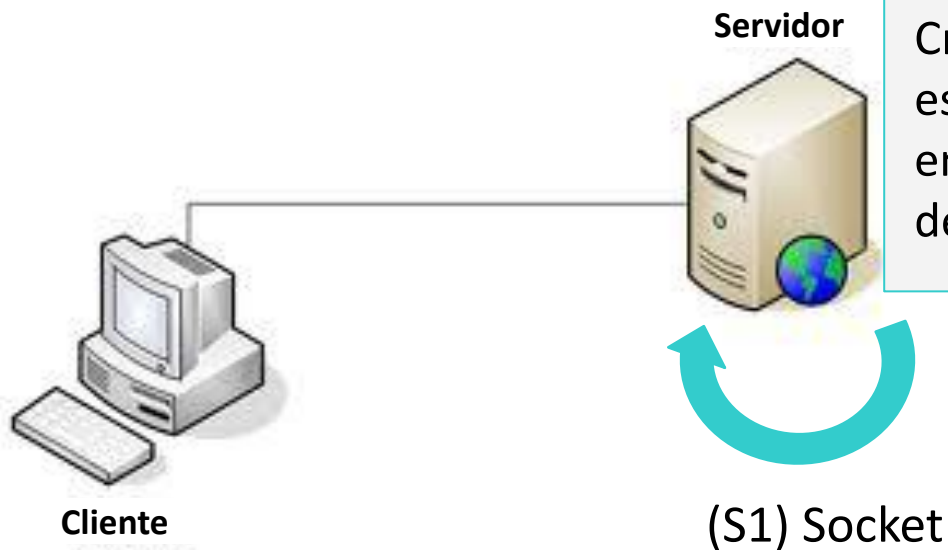
Primitiva	Significado
SOCKET	Criar um novo ponto final de comunicação
BIND	Anexar um endereço local a um soquete
LISTEN	Anunciar a disposição para aceitar conexões; mostrar o tamanho da fila
ACCEPT	Bloquear o responsável pela chamada até uma tentativa de conexão ser recebida
CONNECT	Tentar estabelecer uma conexão ativamente
SEND	Enviar alguns dados através da conexão
RECEIVE	Receber alguns dados da conexão
CLOSE	Encerrar a conexão

Essas primitivas são executadas nesta ordem pelo servidor

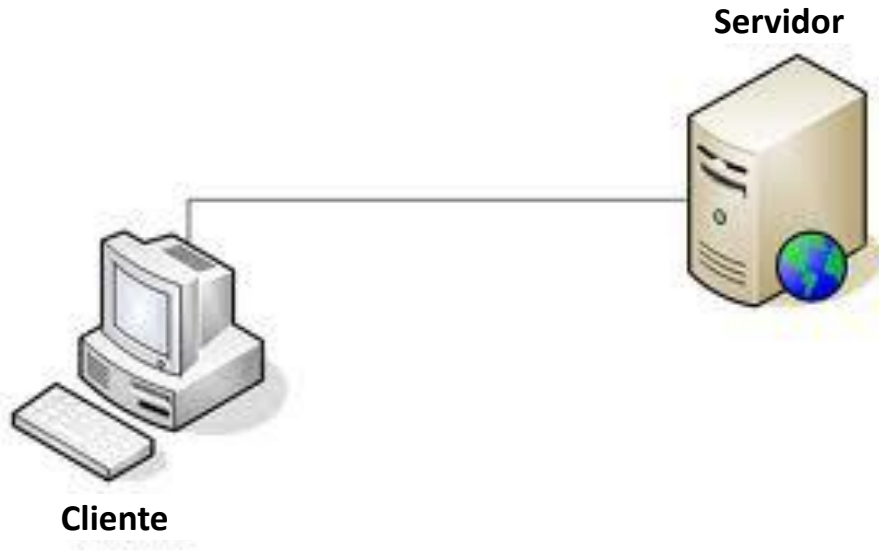
Primitivas de Soquetes para TCP



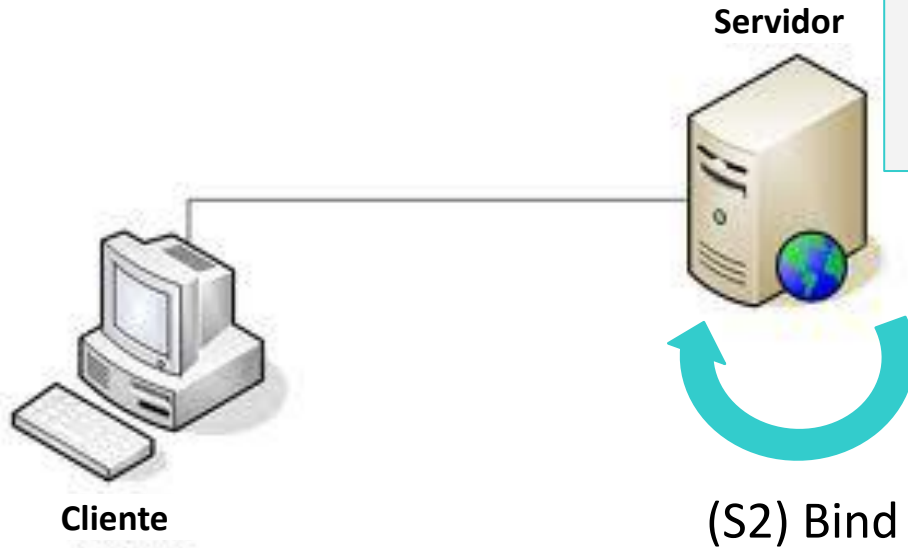
Primitivas de Soquetes para TCP



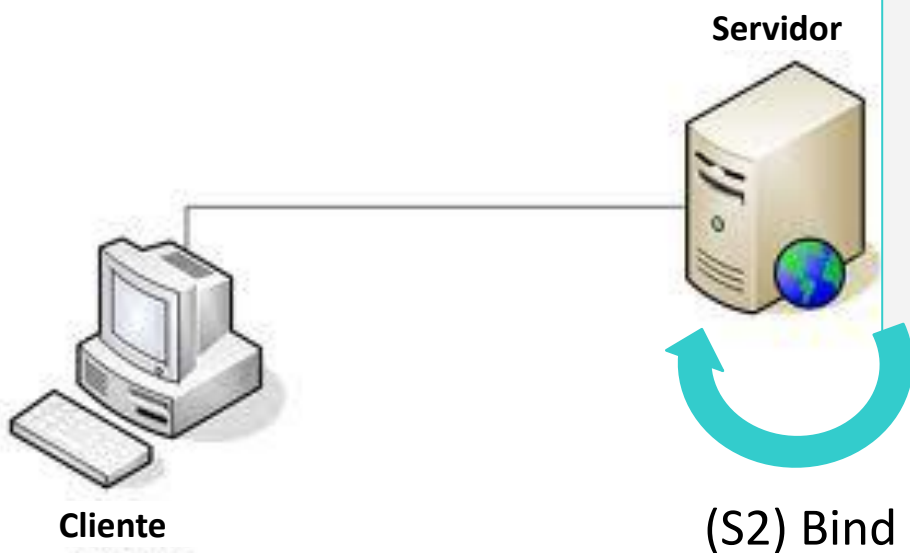
Primitivas de Soquetes para TCP



Primitivas de Soquetes para TCP

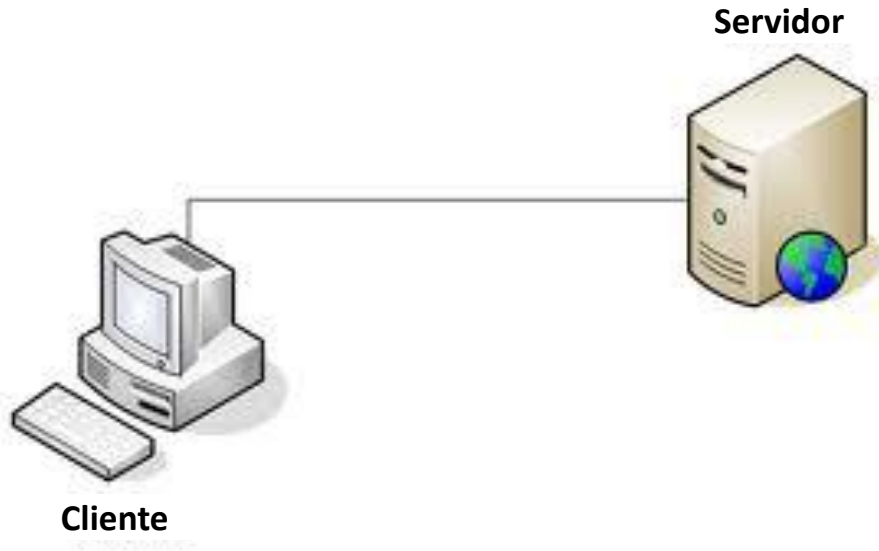


Primitivas de Soquetes para TCP

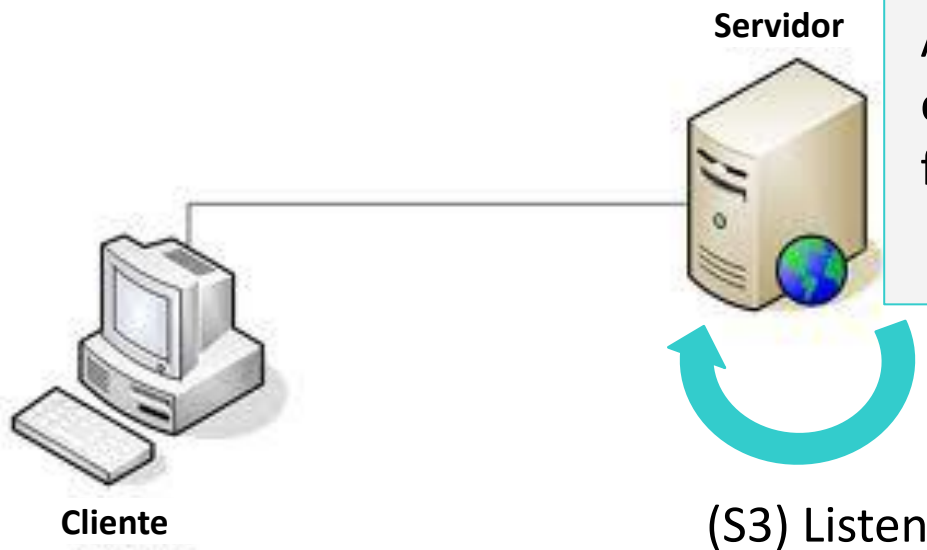


Observação: a primitiva *socket* não atribui diretamente um endereço porque alguns processos utilizam endereços anteriormente definidos

Primitivas de Soquetes para TCP

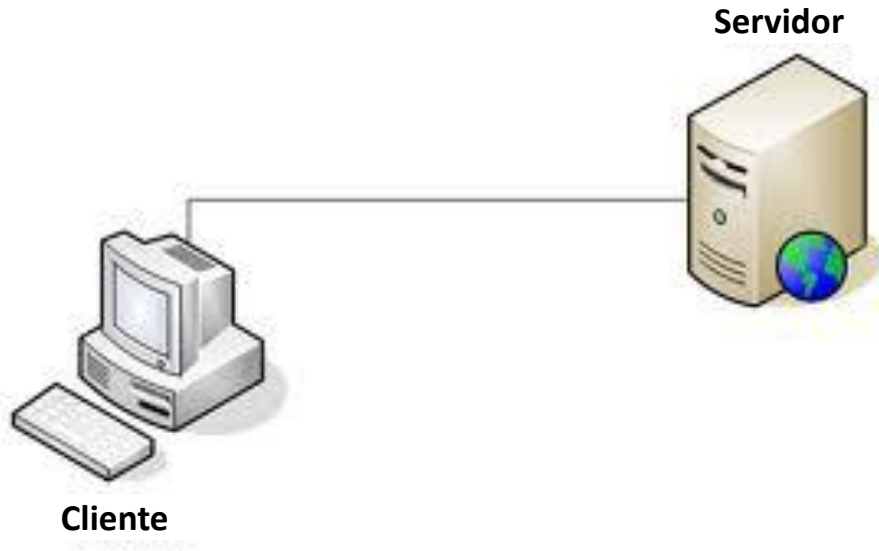


Primitivas de Soquetes para TCP

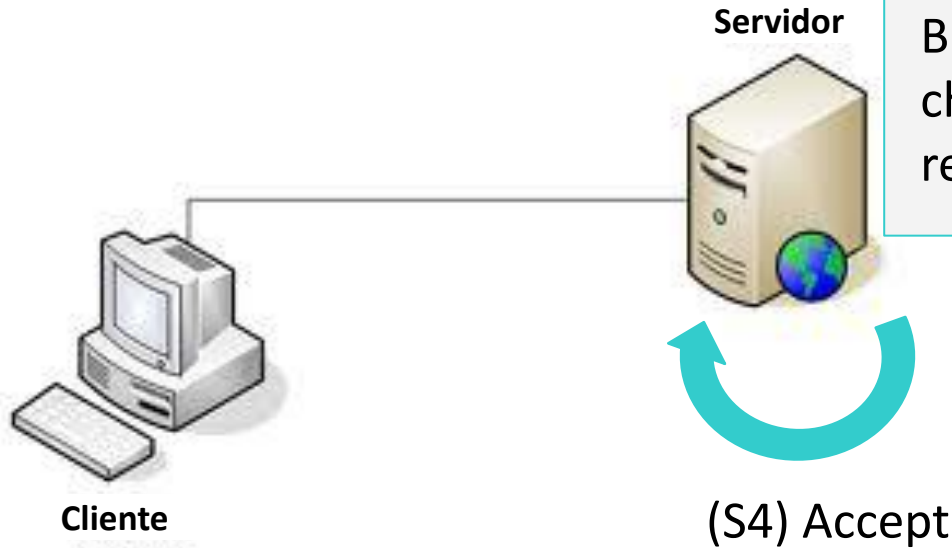


Anuncia a disposição para aceitar conexões e aloca espaço para a fila de chamadas recebidas

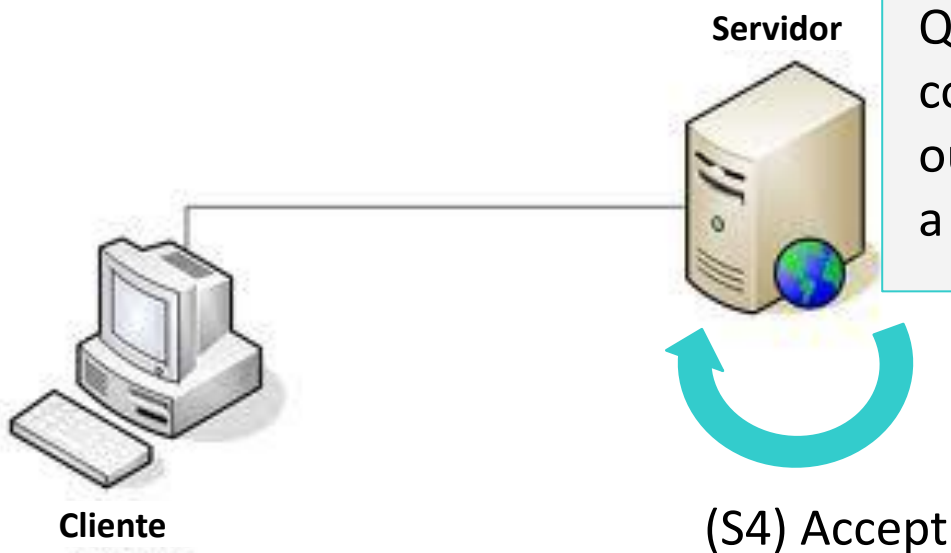
Primitivas de Soquetes para TCP



Primitivas de Soquetes para TCP



Primitivas de Soquetes para TCP

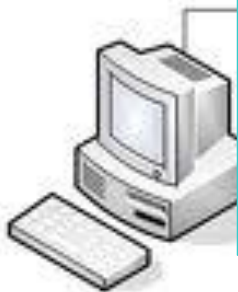


Quando o servidor recebe uma conexão, ele pode desviá-la para outro processo (*thread*) e iniciar a espera por outra conexão

Primitivas de Soquetes para TCP

Servidor

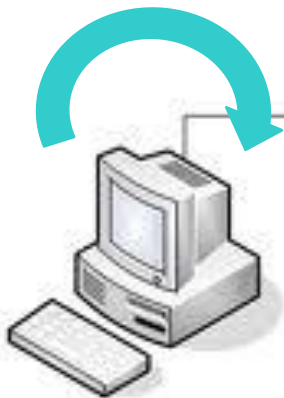
**Vamos para o
lado cliente...**



Cliente

Primitivas de Soquetes para TCP

(C1) Socket



Cliente

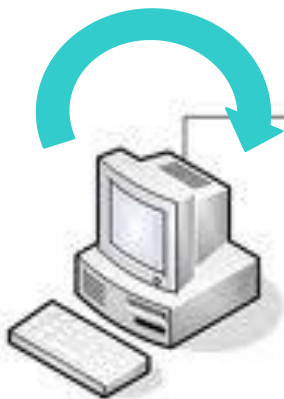
Servidor



Como no servidor, esta primitiva cria um ponto final de comunicação, fazendo as devidas especificações

Primitivas de Soquetes para TCP

(C1) Socket



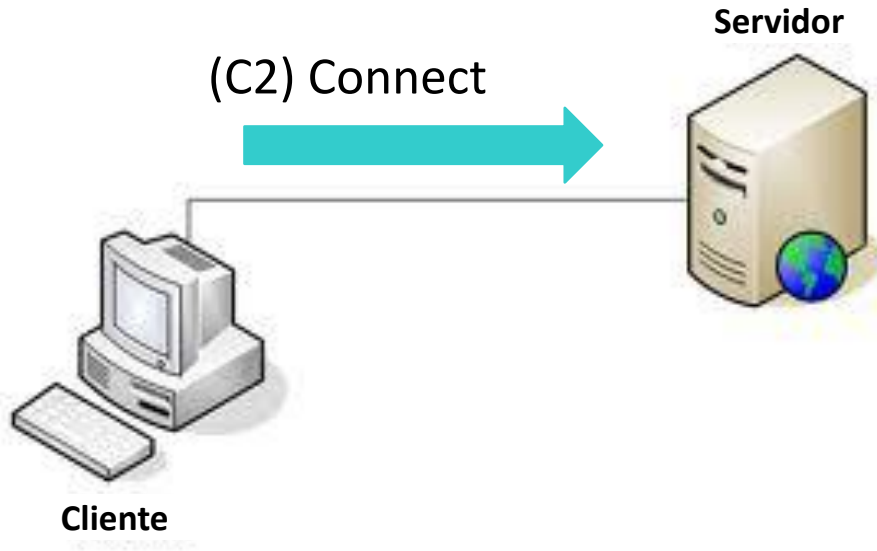
Cliente

Servidor



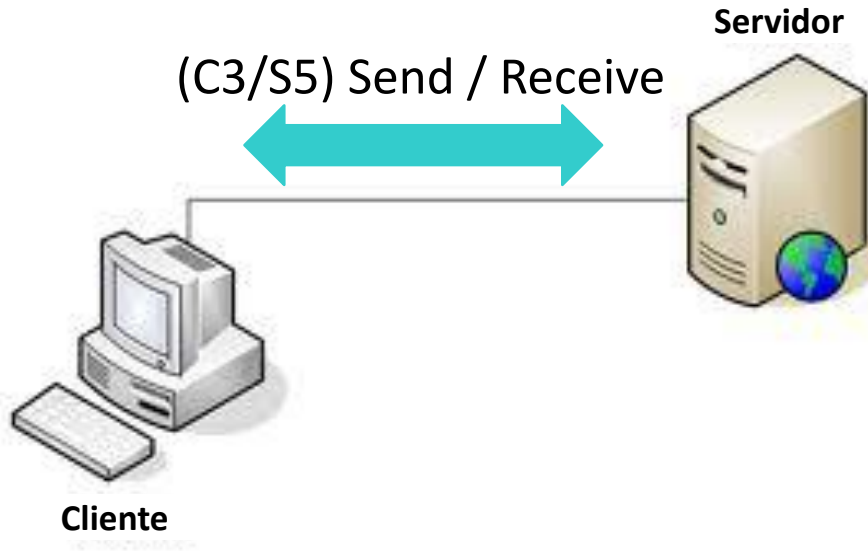
O cliente não efetua a primitiva BIND, pois seu endereço é irrelevante para o servidor

Primitivas de Soquetes para TCP



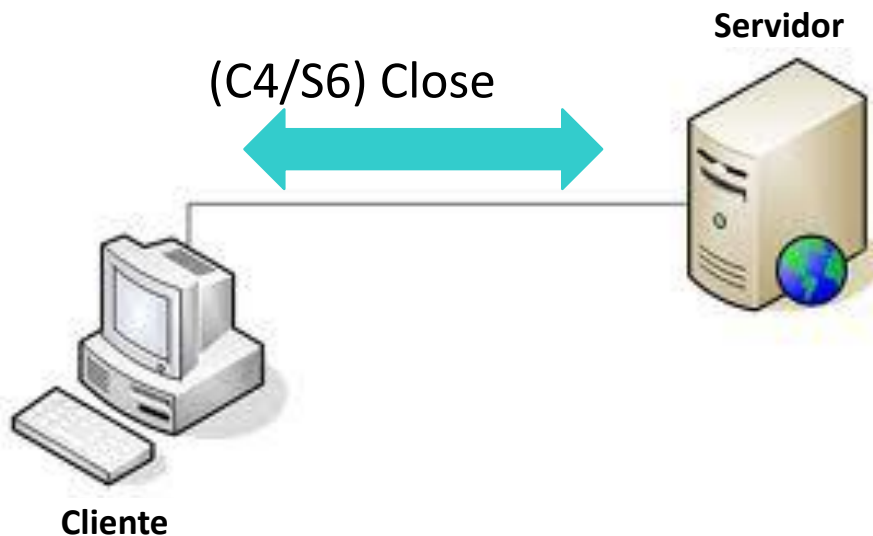
Tenta estabelecer uma conexão com o servidor

Primitivas de Soquetes para TCP



As partes devem ter uma política para definir de quem é a vez de enviar/receber dados

Primitivas de Soquetes para TCP



Desconexão simétrica: cada parte faz sua desconexão para informar que não enviará mais dados

Agenda

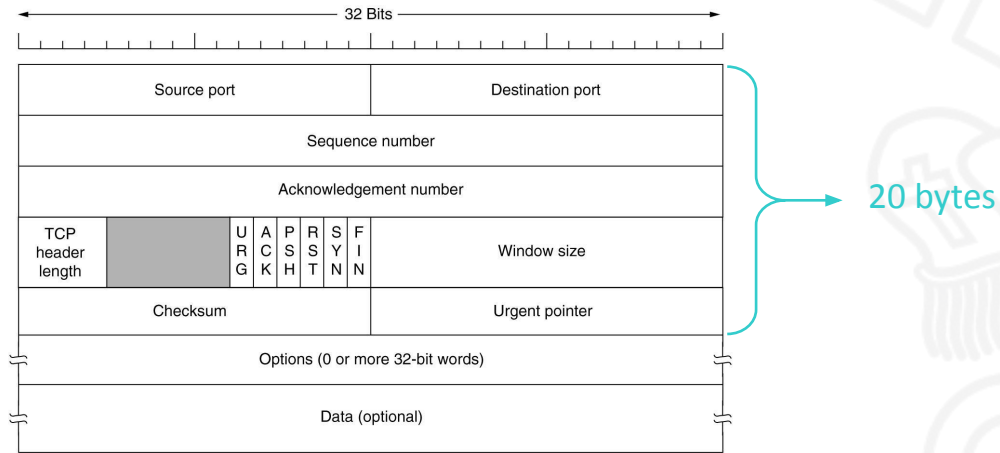
- Introdução
- Primitivas para um Serviço de Transporte Simples
- *User Datagram Protocol (UDP)*
- *Transport Control Protocol (TCP)*

- Introdução
- Primitivas de Soquetes para TCP
- **Cabeçalho do Segmento TCP**
- Serviços do TCP

Tamanho do Segmento TCP

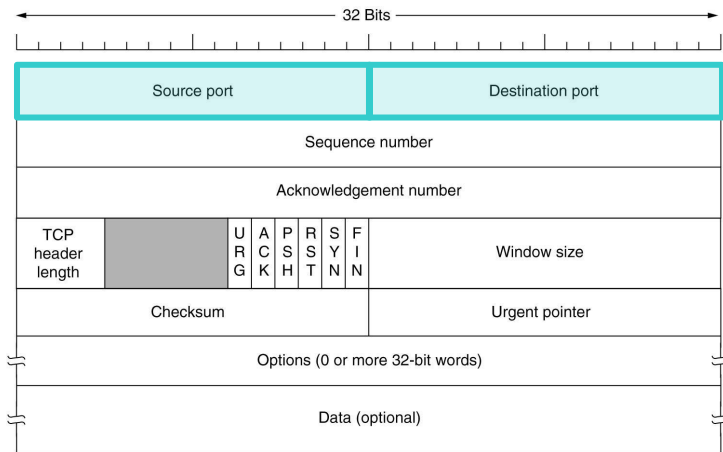
- Normalmente, 1.460 bytes fazendo com que cada segmento caiba em um único quadro Ethernet
- Máximo 64 KB, contudo, isso não acontece na prática

Cabeçalho do Segmento TCP



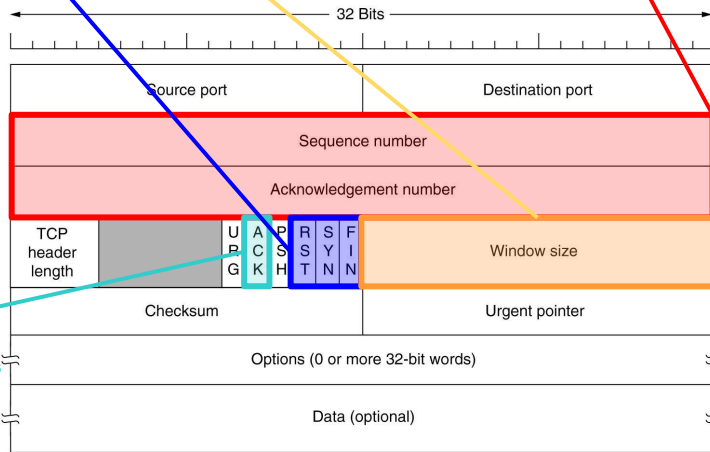
Source Port e Destination Port (16 bits, cada)

- Identificam os processos na origem e destino



Campos para Serviços TCP

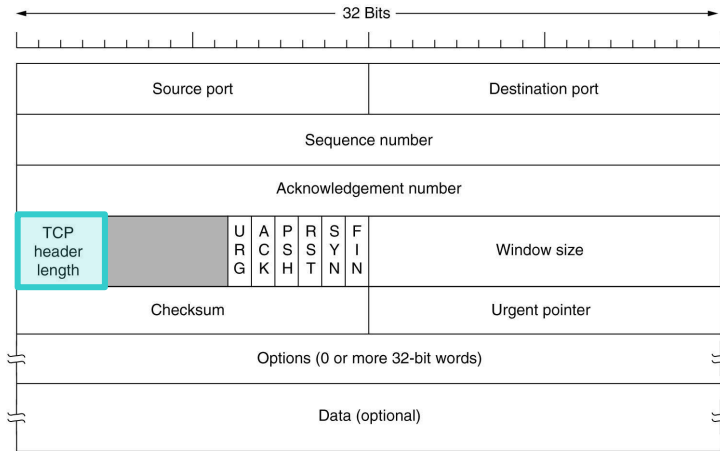
- **Transferência Confiável de Dados**
- **Gerenciamento de Conexões**
- **Controle de Fluxo**



Transferência Confiável de Dados
e Gerenciamento de Conexões

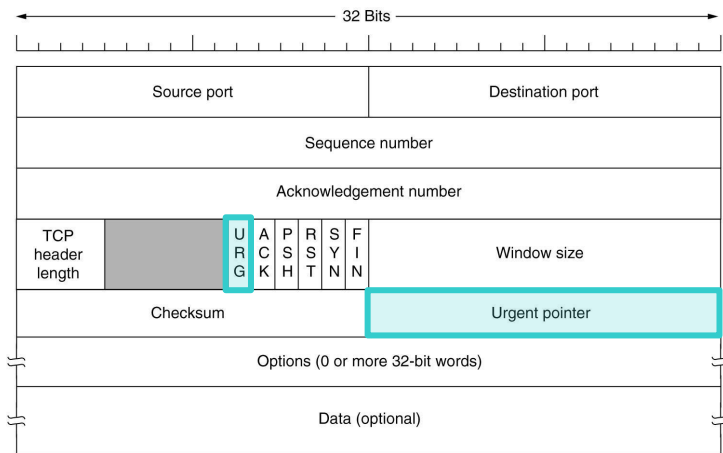
TCP header length (4 bits)

- Informa quantas palavras de 32 bits existem no cabeçalho TCP



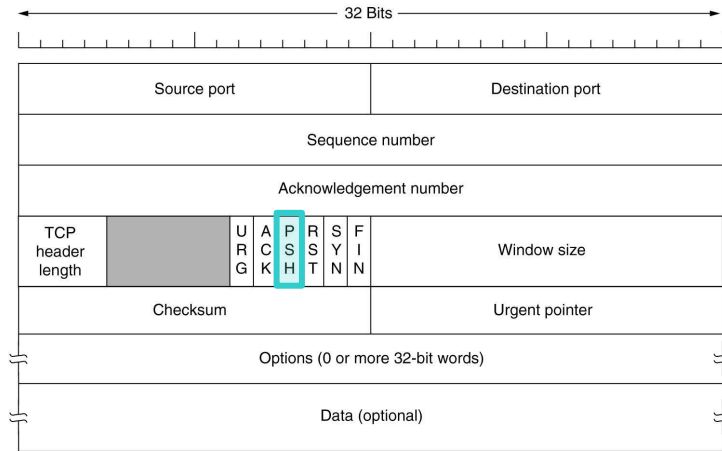
Flag URG (1 bit) e Urgent pointer (16 bits)

- Se URG = 1, Urgent pointer indica o deslocamento de bytes para os dados urgentes (mensagens de interrupção)



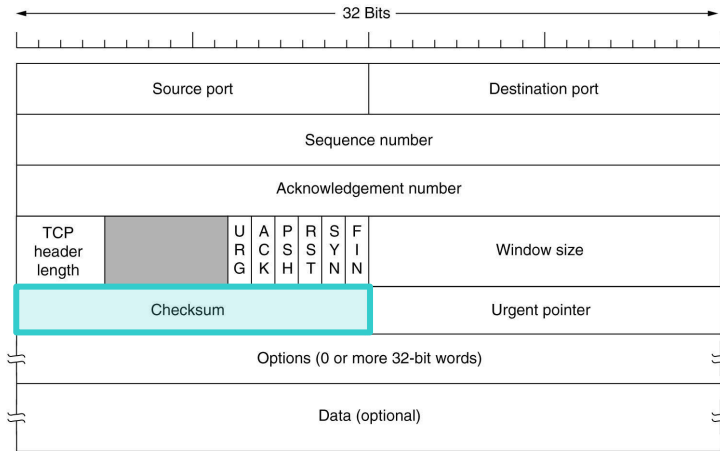
Flag PSH (1 bit)

- Informa ao TCP para não retardar a transmissão
- O receptor é solicitado a entregar os dados à aplicação imediatamente
- Pouco utilizado



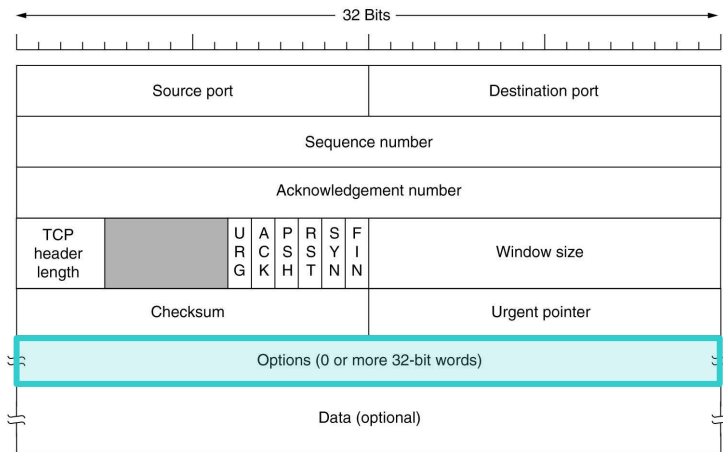
Checksum (16 bits)

- Verificação do segmento



Options (máximo, 40 bytes)

- Permite recursos não previstos no cabeçalho comum (e.g., negociação entre *hosts* para estipular o máximo de carga útil do TCP)



Agenda

- Introdução
- Primitivas para um Serviço de Transporte Simples
- *User Datagram Protocol (UDP)*
- ***Transport Control Protocol (TCP)***
 - Introdução
 - Primitivas de Soquetes para TCP
 - Cabeçalho do Segmento TCP
 - **Serviços do TCP**
 - Transferência Confiável de Dados
 - Gerenciamento da Conexão TCP
 - Controle de Fluxo
 - Controle de Congestionamento
 - Gerenciamento de Temporizadores

Agenda

- Introdução
- Primitivas para um Serviço de Transporte Simples
- *User Datagram Protocol (UDP)*
- ***Transport Control Protocol (TCP)***
 - Introdução
 - Primitivas de Soquetes para TCP
 - Cabeçalho do Segmento TCP
 - **Serviços do TCP**
 - **Transferência Confiável de Dados**
 - Gerenciamento da Conexão TCP
 - Controle de Fluxo
 - Controle de Congestionamento
 - Gerenciamento de Temporizadores

Transferência Confiável de Dados

- Provê um serviço de entrega de dados confiável para as aplicações
- Trata perdas e atrasos sem sobrecarregar a rede e seus roteadores
- Garante a cadeia de bytes recebida no destino:
 - É exatamente a mesma enviada pela origem
 - Não está corrompida
 - Não tem lacunas
 - Não possui duplicações
 - Está em sequência

Transferência Confiável de Dados

- Baseada em confirmações utilizando o Flag ACK



Entidade TCP
Emissora



Entidade TCP
Receptora

Transferência Confiável de Dados

- Baseada em confirmações utilizando o Flag ACK

(1) Emissor envia um segmento e dispara um temporizador



Transferência Confiável de Dados

- Baseada em confirmações utilizando o Flag ACK

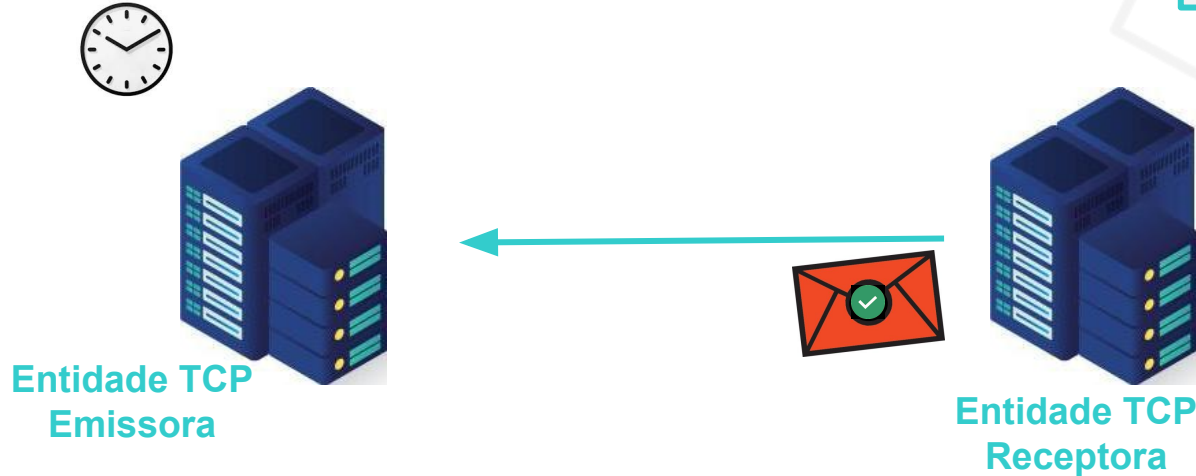
(2) Receptor recebe o segmento



Transferência Confiável de Dados

- Baseada em confirmações utilizando o Flag ACK

(3) Receptor envia um segmento ACK



Transferência Confiável de Dados

- Baseada em confirmações utilizando o Flag ACK

(4) Quando o emissor recebe o segmento ACK, ele cancela o temporizador



Entidade TCP
Emissora



Entidade TCP
Receptora

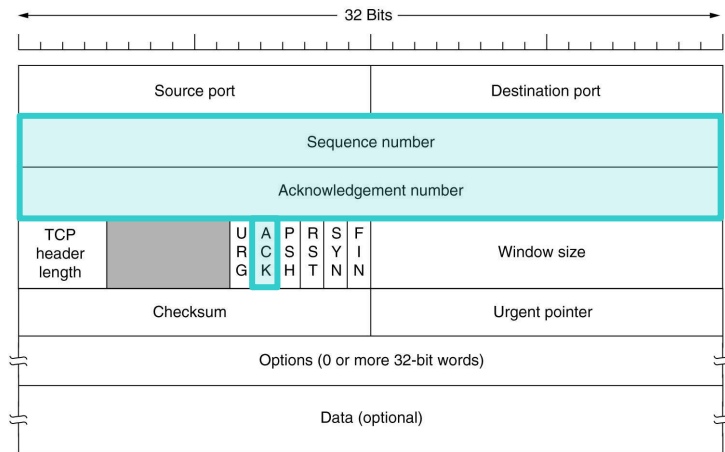
Transferência Confiável de Dados

- Baseada em confirmações utilizando o Flag ACK

(5) Se o temporizador expirar e o emissor não tiver recebido o ACK, ele retransmite o segmento

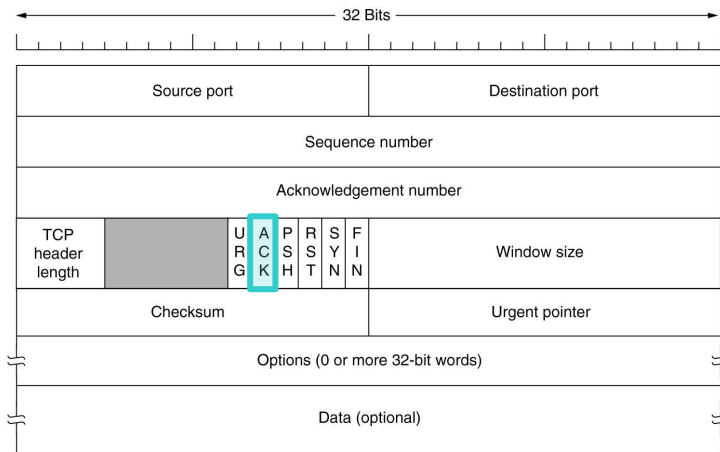


Campos para Transferência Confiável de Dados



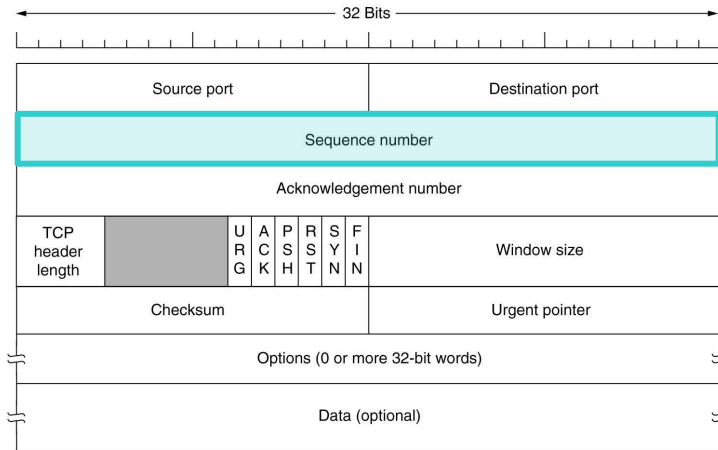
Flag ACK (1 bit)

- Se ACK = 1, o campo *Acknowledgement number* é válido, senão, o segmento não contém uma confirmação

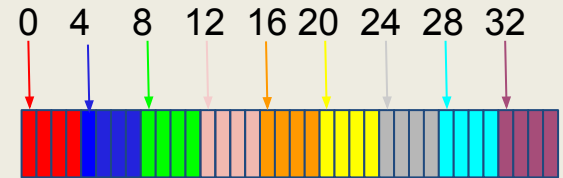


Campo *Sequence number* (32 bits)

- Indica o número do primeiro byte de dados do segmento no fluxo de dados a ser transmitido

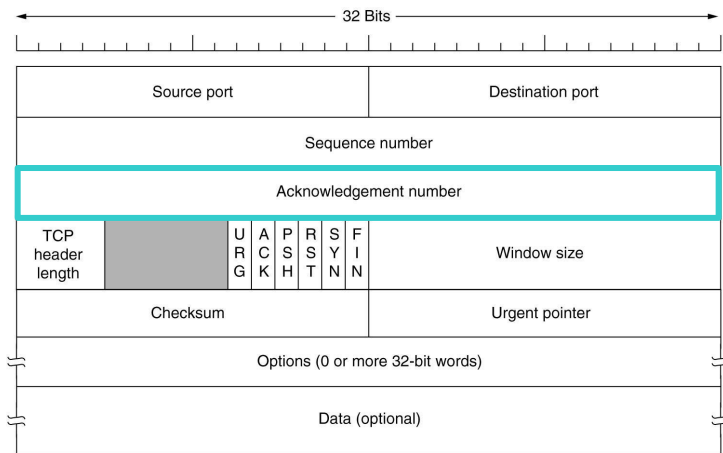


Exemplo de fluxo de dados a ser transmitido em 9 segmentos de 4 bytes cada. Abaixo, veja o *sequence number* de cada segmento

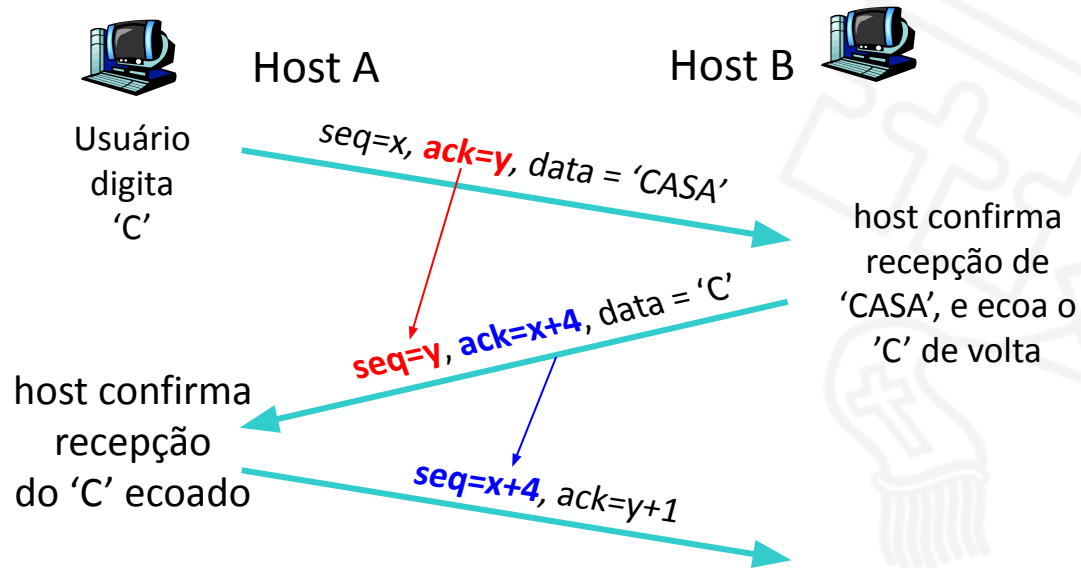


Campo *Acknowledgement number* (32 bits)

- Especifica o número do próximo byte esperado (não o último byte recebido corretamente)



Exemplo em um Cenário Telnet Simple



Agenda

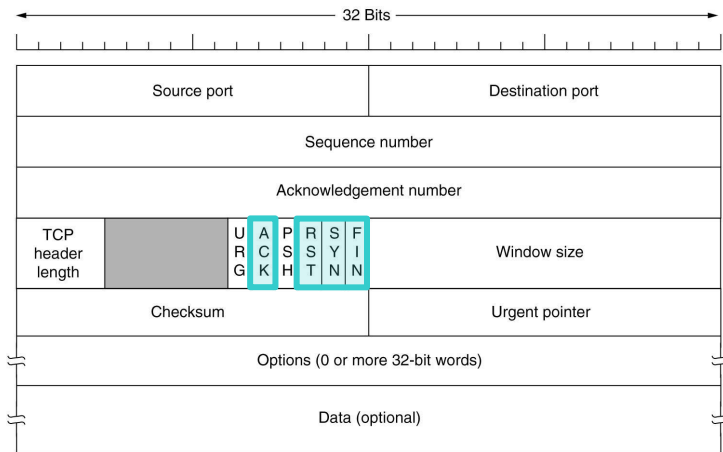
- Introdução
- Primitivas para um Serviço de Transporte Simples
- *User Datagram Protocol (UDP)*
- ***Transport Control Protocol (TCP)***
 - Introdução
 - Primitivas de Soquetes para TCP
 - Cabeçalho do Segmento TCP
 - **Serviços do TCP**
 - Transferência Confiável de Dados
 - **Gerenciamento da Conexão TCP**
 - Controle de Fluxo
 - Controle de Congestionamento
 - Gerenciamento de Temporizadores

Conexões do TCP

- São *full-duplex*, logo, o tráfego de dados pode ser feito em ambas as direções ao mesmo tempo
- São *ponto a ponto*, logo, cada conexão possui exatamente dois pontos terminais, não admitindo multidifusão e difusão

Campos para Gerenciamento de Conexões

- **Flag SYN**: Estabelece conexões
- **Flag RST**: Recusa uma tentativa de conexão
- **Flag FIN**: Finaliza uma conexão



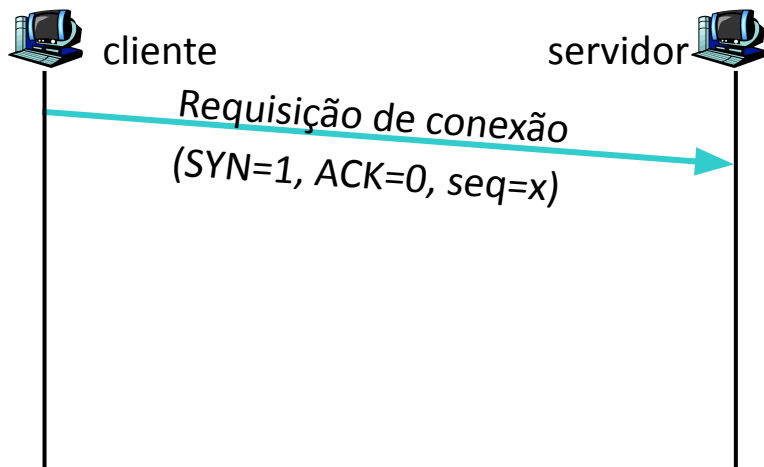
Flag ACK: Faz confirmações

Gerenciamento da Conexão TCP

- A entidade TCP emissora estabelece uma conexão com a receptora antes de trocar segmentos de dados
- As conexões são estabelecidas através do *3-way handshake* (3 vias)
- A conexão inicializa variáveis:
 - Números de sequência
 - *Buffers*, controle de fluxo

Estabelecimento da Conexão: *3-way handshake*

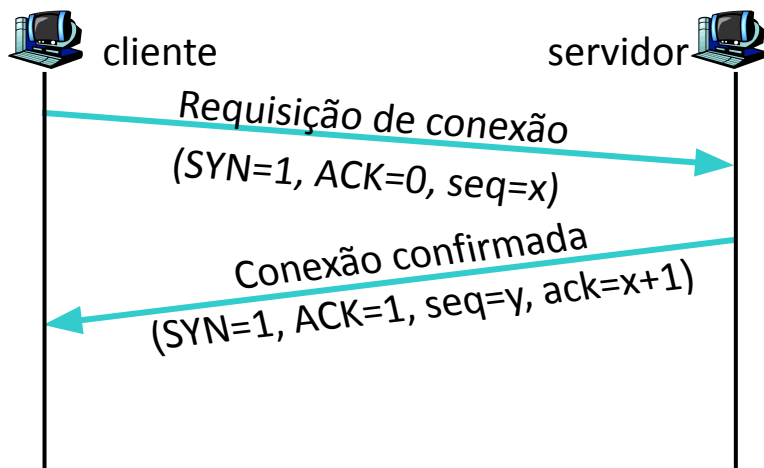
- **Passo 1)** Cliente envia um segmento com $\text{SYN}=1$ e $\text{ACK}=0$ ao servidor para especificar o número inicial da sequência



Um segmento SYN consome 1 byte de espaço de sequência

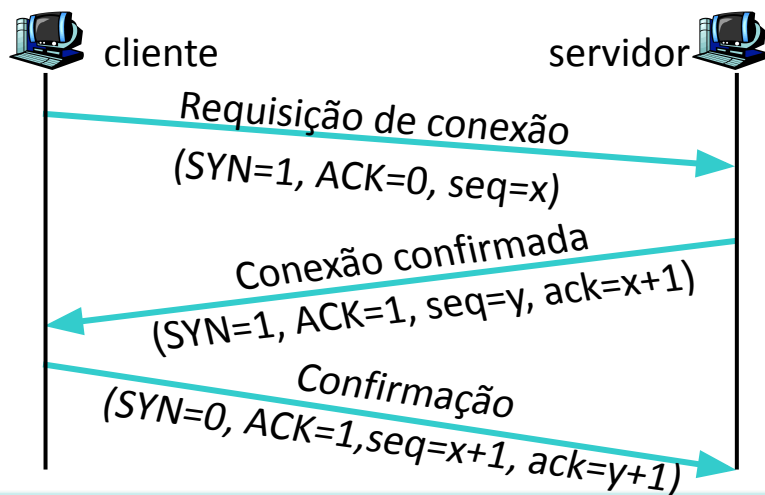
Estabelecimento da Conexão: 3-way handshake

- **Passo 2)** Servidor responde com o segmento SYNACK (SYN=1, ACK=1), reconhecendo o SYN recebido, alocando *buffers* e especificando o número de inicial de sua sequência



Estabelecimento da Conexão: 3-way handshake

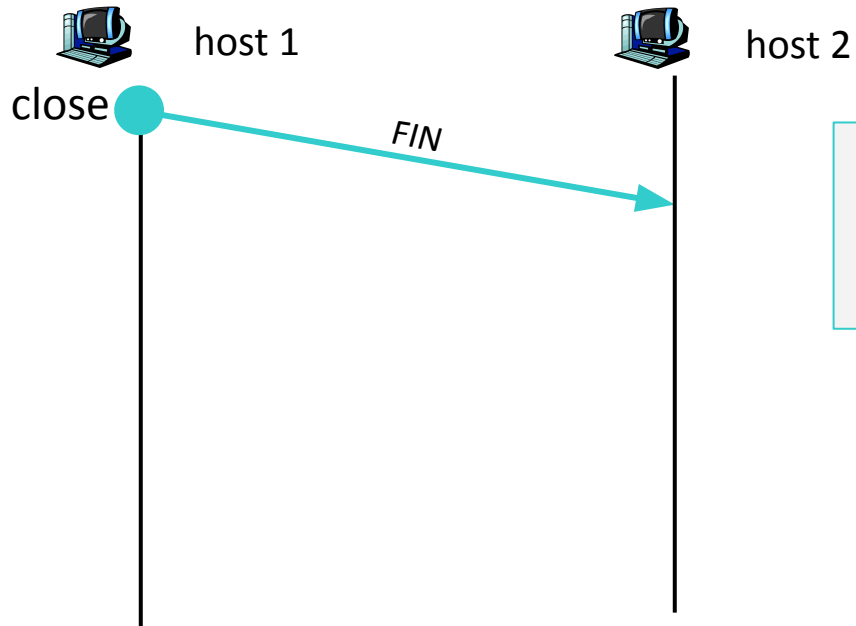
- **Passo 3)** Cliente recebe o SYNACK e envia um ACK



Término da Conexão TCP

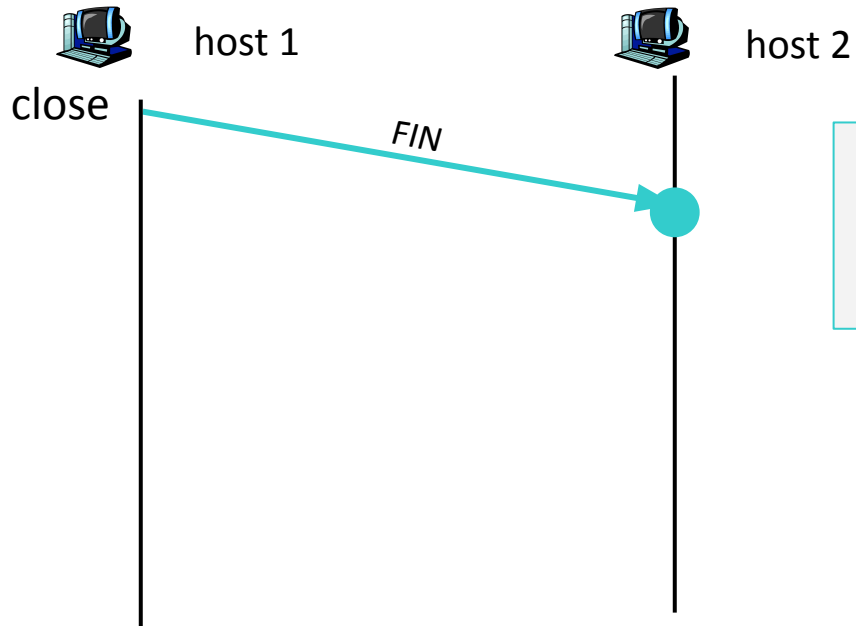
- Apesar das conexões TCP serem *full-duplex*, podemos compreender seu encerramento como se cada conexão fosse um par de conexões *simplex*
- Assim, no TCP, cada uma dessas conexões “*simplex*” é encerrada de modo independente de sua parceira

Término da Conexão TCP



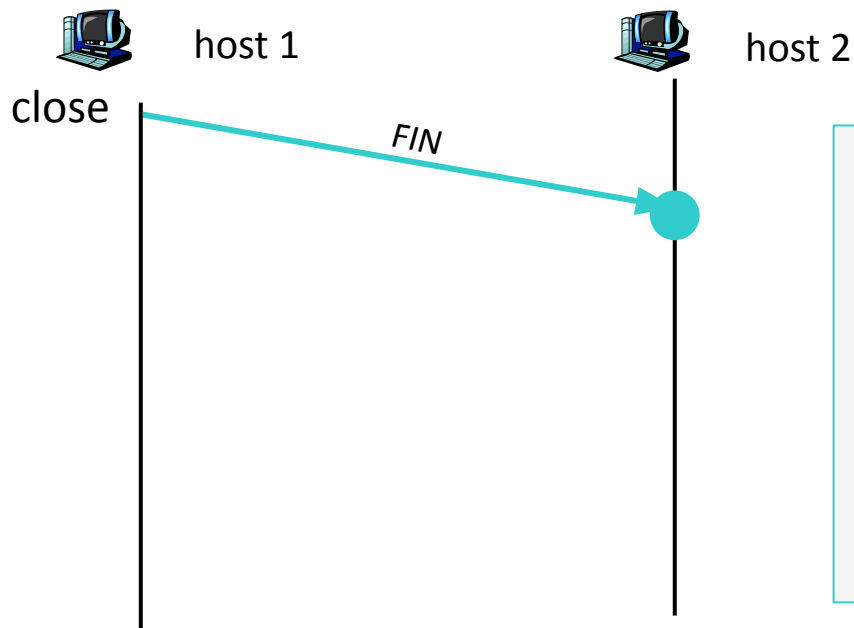
Passo 1) O *host 1* envia um segmento TCP FIN para o *host 2*

Término da Conexão TCP



Passo 2) O *host 2* recebe o FIN

Término da Conexão TCP

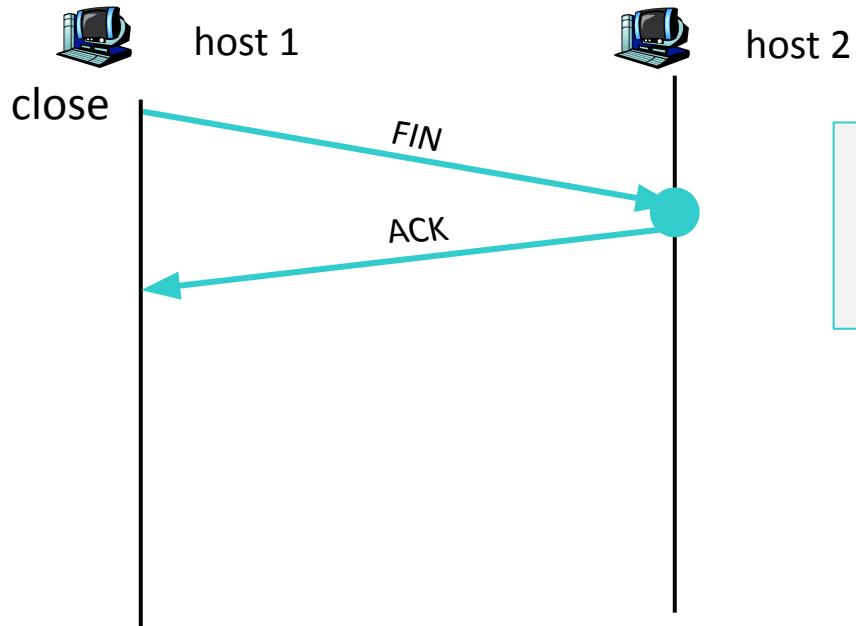


Passo 2) O *host 2* recebe o FIN

Assim, o *host 2* fecha a conexão no sentido *host 1* $\Rightarrow$ *host 2*

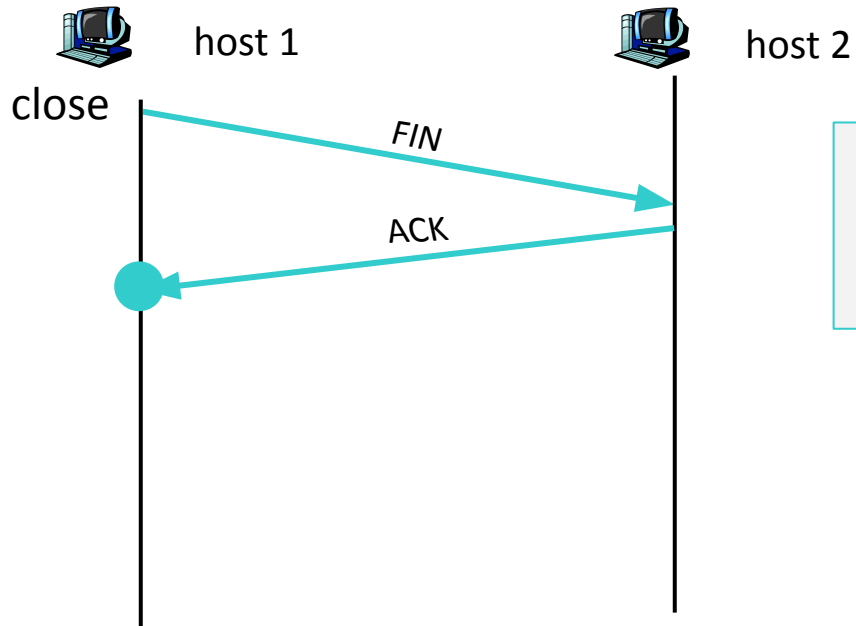
Contudo, o fluxo de dados pode continuar no sentido oposto

Término da Conexão TCP



Passo 3) O *host 2* responde com um ACK

Término da Conexão TCP



Passo 4) O *host 1* recebe o ACK e fecha a conexão no sentido *host 1* $\Rightarrow$ *host 2*

Término da Conexão TCP



host 1

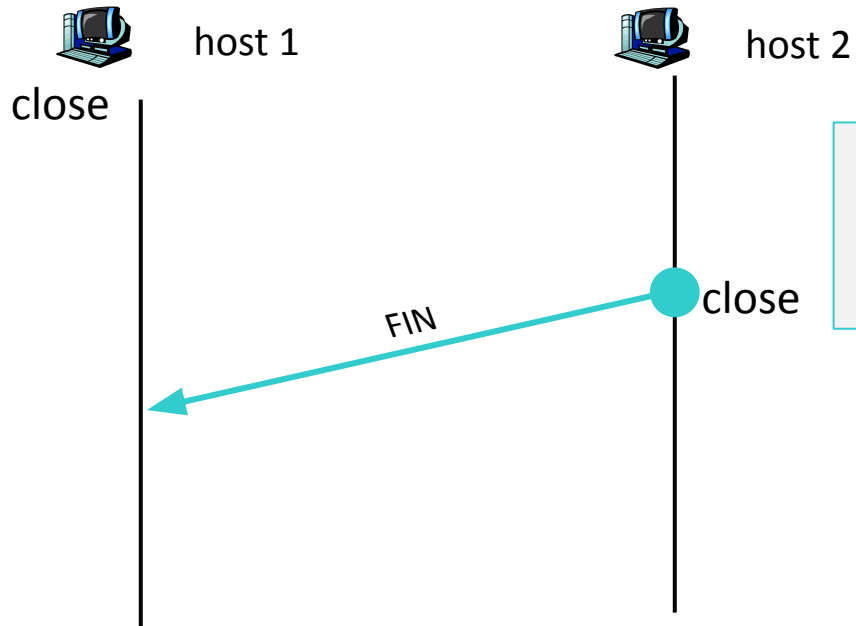


host 2

close

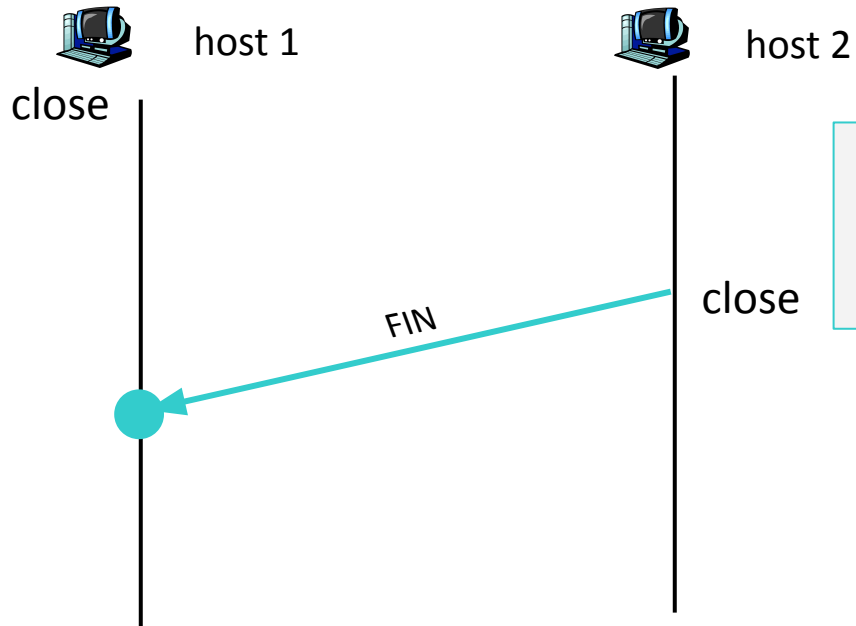
Após um tempo...

Término da Conexão TCP



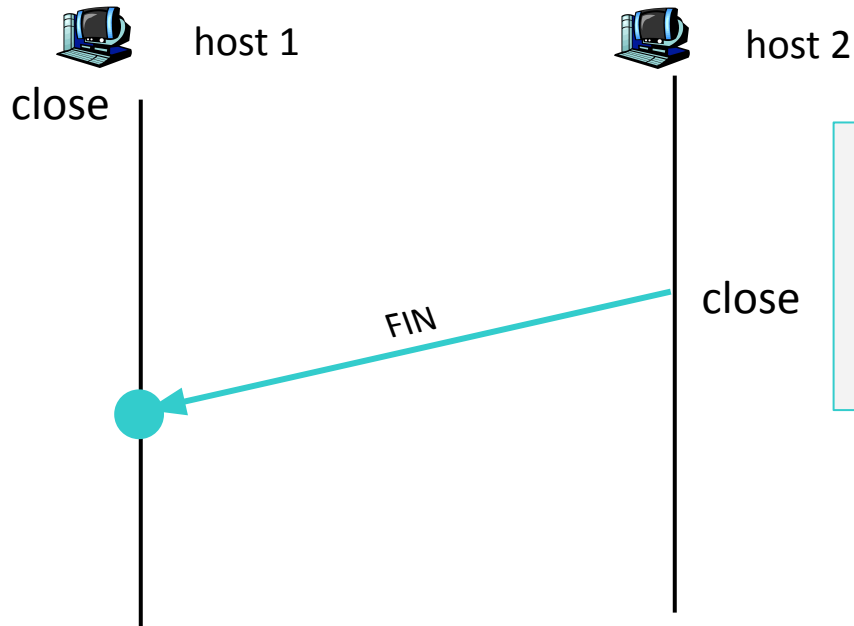
Passo 5) Em algum momento, o *host 2* envia o segmento TCP FIN para o *host 1*

Término da Conexão TCP



Passo 6) O *host 1* recebe o FIN

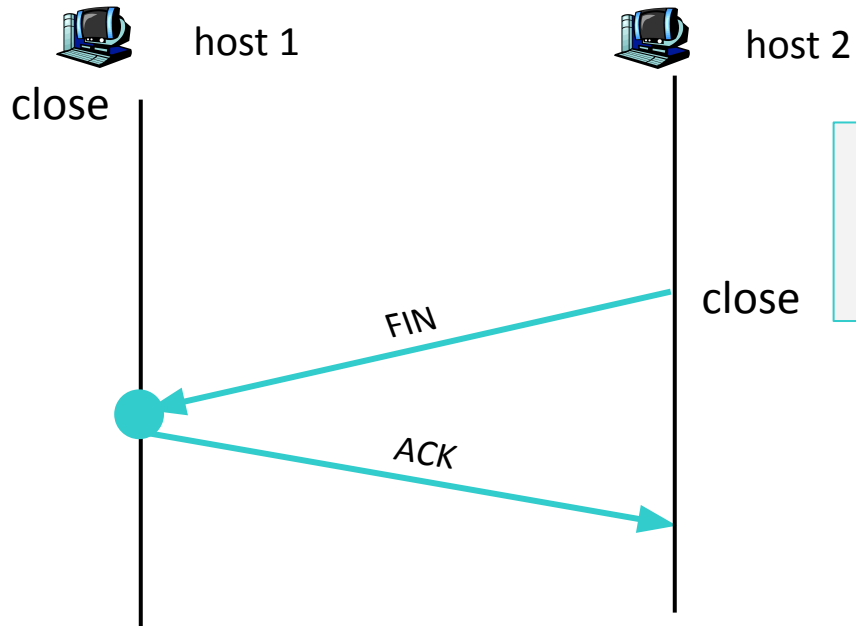
Término da Conexão TCP



Passo 6) O *host 1* recebe o FIN

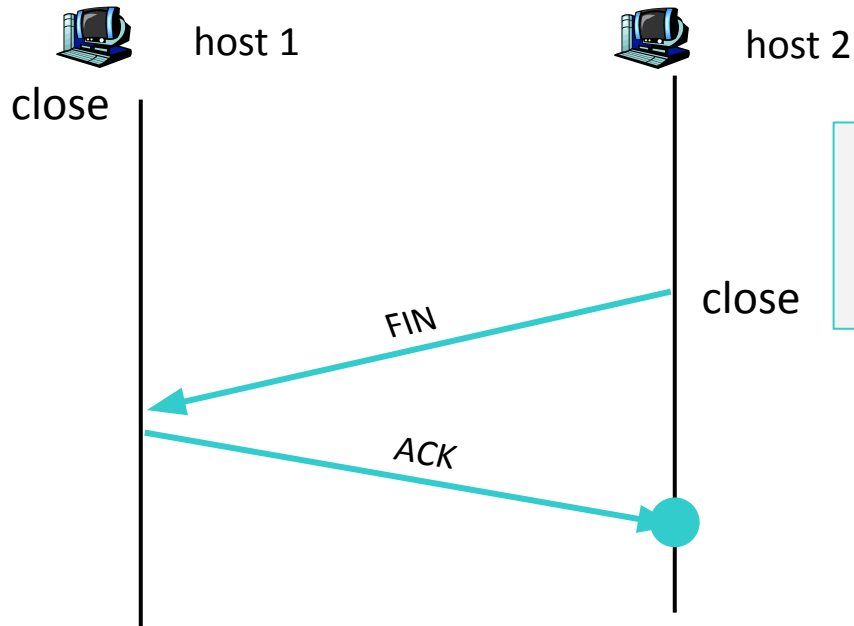
Assim, o *host 1* fecha a conexão no sentido *host 2* ⇒ *host 1*

Término da Conexão TCP



Passo 7) O *host 1* responde com um ACK

Término da Conexão TCP



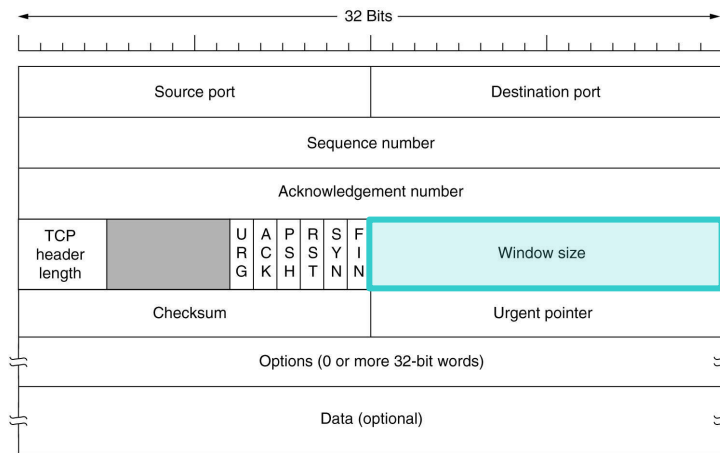
Passo 8) O *host 2* recebe o ACK e fecha a conexão no sentido *host 2* $\Rightarrow$ *host 1*

Agenda

- Introdução
- Primitivas para um Serviço de Transporte Simples
- *User Datagram Protocol (UDP)*
- ***Transport Control Protocol (TCP)***
 - Introdução
 - Primitivas de Soquetes para TCP
 - Cabeçalho do Segmento TCP
 - **Serviços do TCP**
 - Transferência Confiável de Dados
 - Gerenciamento da Conexão TCP
 - **Controle de Fluxo**
 - Controle de Congestionamento
 - Gerenciamento de Temporizadores

Controle de Fluxo

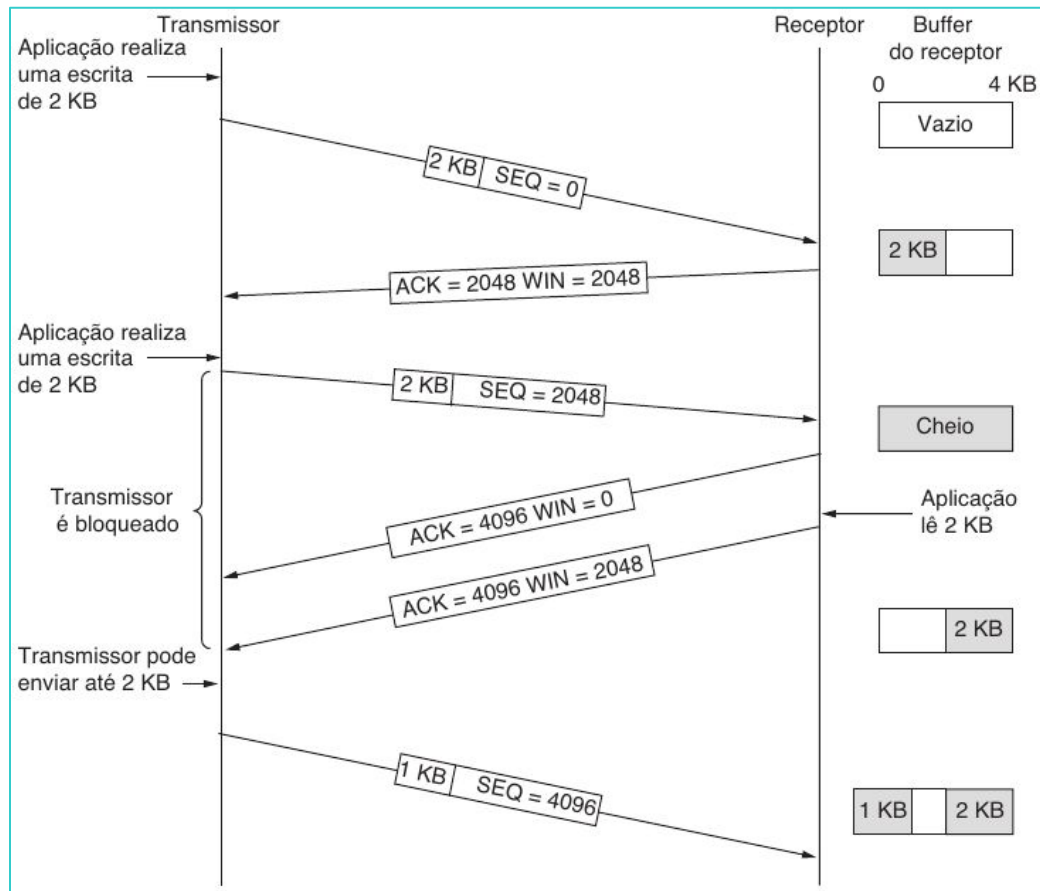
- Evitar que o emissor esgote a capacidade do receptor
- Utiliza o campo *Window size* do cabeçalho TCP



Funcionamento do Controle de Fluxo

- No estabelecimento da conexão, cada entidade TCP aloca um *buffer* de recepção e informa o tamanho desse *buffer* para a outra entidade
- Em toda confirmação, envia-se o espaço disponível nesse *buffer* e esse espaço é chamado de janela
- Conhecendo a janela de seu par, uma entidade TCP sabe a quantidade de bytes que a outra entidade TCP pode receber
- O tamanho do *buffer* e a janela estão disponíveis no campo *Window size*

Exemplo de Gerenciamento de Janelas no TCP



Agenda

- Introdução
- Primitivas para um Serviço de Transporte Simples
- *User Datagram Protocol (UDP)*
- ***Transport Control Protocol (TCP)***
 - Introdução
 - Primitivas de Soquetes para TCP
 - Cabeçalho do Segmento TCP
 - **Serviços do TCP**
 - Transferência Confiável de Dados
 - Gerenciamento da Conexão TCP
 - Controle de Fluxo
 - **Controle de Congestionamento**
 - Gerenciamento de Temporizadores

Controle de Congestionamento

- Quando a carga oferecida a qualquer rede é maior que sua capacidade, acontece um congestionamento
- Diferente de controle de fluxo!
- Sintomas:
 - Perda de pacotes (saturação de *buffer* nos roteadores)
 - Atrasos grandes (filas nos *buffers* dos roteadores)
- Um dos 10 problemas mais importantes na Internet!

Considerações sobre o Controle de Congestionamento

- O congestionamento da rede pode ser piorado se a camada de transporte retransmite pacotes que não foram perdidos
- Esse problema pode causar até um colapso da rede

Como detectar congestionamento?

- TCP usa a quantidade de pacotes perdidos (pacotes com *timeout*) como uma medida de congestionamento
- TCP reduz a taxa de retransmissão à medida que esse valor aumenta

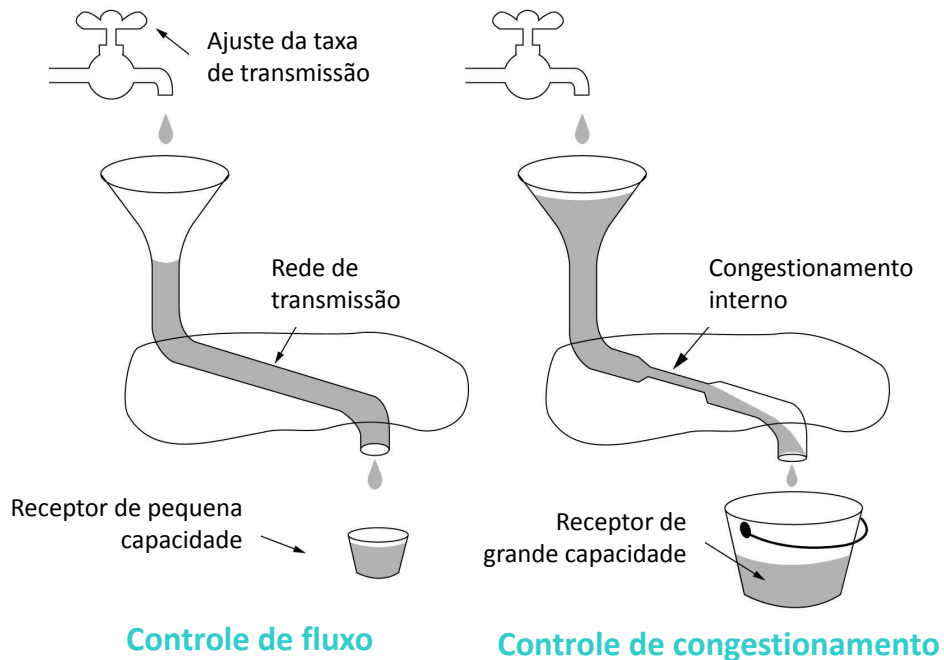
Considerações sobre o Controle de Congestionamento

- Antigamente, um *timeout* causado por um pacote perdido podia ter sido provocado por
 - Ruído em uma linha de transmissão, ou
 - O pacote foi descartado em um roteador congestionado
- Hoje em dia, a perda de pacotes devido a erros de transmissão é relativamente rara
- A maioria dos *timeouts* de transmissão na Internet se deve a congestionamentos

Dois Problemas em Potenciais na Internet

- **Capacidade da rede** $\Rightarrow$ Controle de Congestionamento
- **Capacidade do receptor** $\Rightarrow$ Controle de Fluxo

Controle de Fluxo vs. de Congestionamento



Número de Bytes que podem ser Transmitidos

- Cada emissor mantém duas janelas:
 - Janela fornecida pelo receptor (controle de fluxo)
 - Janela de congestionamento (controle de congestionamento)
- O número de bytes que podem ser transmitidos é o valor mínimo entre as duas janelas

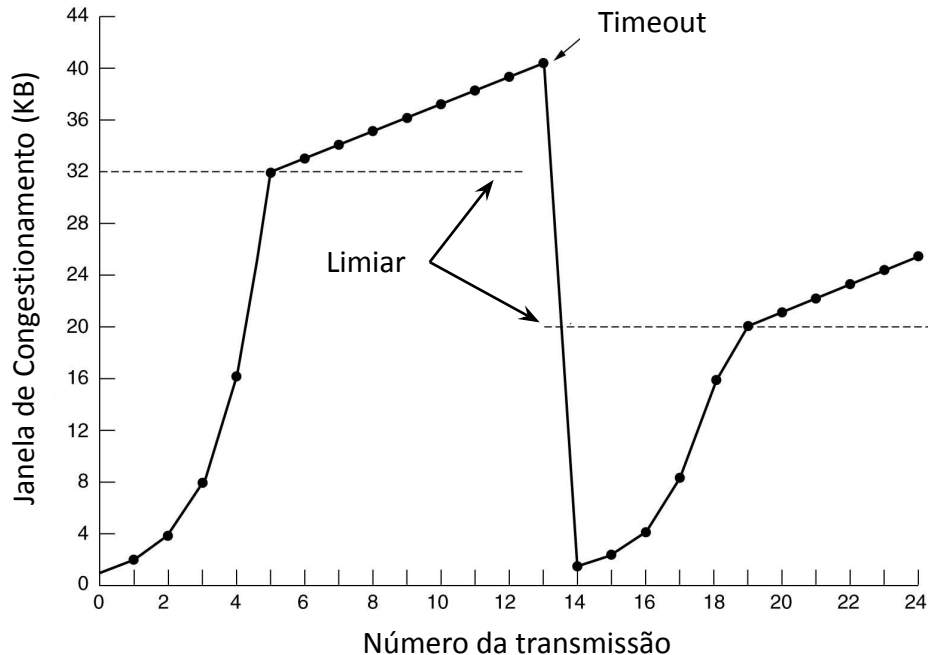
Algoritmo de Iniciação Lenta

- Quando uma conexão é estabelecida, a janela de congestionamento é igual ao tamanho do segmento máximo em uso na conexão
- Cada rajada confirmada duplica a janela de congestionamento
- O crescimento exponencial da janela de congestionamento acontece até que:
 - Um limiar seja atingido,
 - Aconteça um *timeout* indicando a retransmissão de um segmento, ou
 - A janela do receptor seja alcançada

Término do Crescimento Exponencial

- **Um limiar é atingido:** as transmissões bem-sucedidas proporcionam um crescimento linear à janela de congestionamento
- **Acontece um *timeout* indicando a retransmissão de um segmento:**
 - O limiar é definido como a metade da janela de congestionamento atual
 - Janela de congestionamento $\Rightarrow$ segmento máximo
 - Inicialização lenta é usada
- **Janela de recepção é alcançada:** janela de congestionamento pára de crescer e permanece constante

Término do Crescimento Exponencial



A transmissão de mensagens é feita de forma exponencial até atingir um limiar, quando ela passa a aumentar mais lentamente

Agenda

- Introdução
- Primitivas para um Serviço de Transporte Simples
- *User Datagram Protocol (UDP)*
- ***Transport Control Protocol (TCP)***
 - Introdução
 - Primitivas de Soquetes para TCP
 - Cabeçalho do Segmento TCP
 - **Serviços do TCP**
 - Transferência Confiável de Dados
 - Gerenciamento da Conexão TCP
 - Controle de Fluxo
 - Controle de Congestionamento
 - **Gerenciamento de Temporizadores**

Gerenciamento de Temporizadores

- Quando um segmento é enviado, um *timer* de retransmissão é ativado
- Se o segmento é confirmado antes do término do *timer*, ele é interrompido
- Se o *timer* expirar antes da confirmação chegar, o segmento é retransmitido e o *timer*, disparado novamente

Tempo de Viagem de Ida e Volta (RTT)

- Do inglês, *Round Trip Time*
- Tempo transcorrido desde o instante em que um segmento é enviado até o instante em que ele é reconhecido
- Para cada conexão, o TCP mantém uma variável, RTT, que é a melhor estimativa no momento para o tempo de percurso de ida e volta até o destino em questão

Como Escolher o Valor do *Timer* do TCP?

- Maior que o RTT (RTT varia)
- Muito curto: Retransmissões desnecessárias, sobrecarregando a Internet com pacotes inúteis
- Muito longo: O desempenho será prejudicado devido ao longo retardo de retransmissão sempre que um pacote se perder

Como Escolher o Valor do *Timer* do TCP?

- O TCP atualiza a variável RTT de acordo com a fórmula:

$$RTT = \alpha RTT + (1 - \alpha)M$$

onde:

- α : fator de suavização que determina o peso dado ao antigo valor
- M: último RTT medido

Exercícios

Exercício (1)

- Faça um paralelo entre a porta para a camada de transporte e o endereço de rede para a camada de rede.

Exercício (2)

- Por que a camada de transporte dá sentido à pilha de protocolos?

Exercício (3)

- O que significa uma aplicação do tipo *one shot*? O protocolo TCP pode ser utilizado na camada de transporte para esse tipo de aplicação? E o UDP? Justifique suas respostas.

Exercício (4)

- Explique como executamos as primitivas de sockets para o TCP em uma aplicação do tipo cliente-servidor

Exercício (5)

- Descreva a transferência confiável de dados do TCP e mostre como os campos *Sequence* e *Acknowledgement Numbers* e *ACK* atuam nesse caso

Exercício (6)

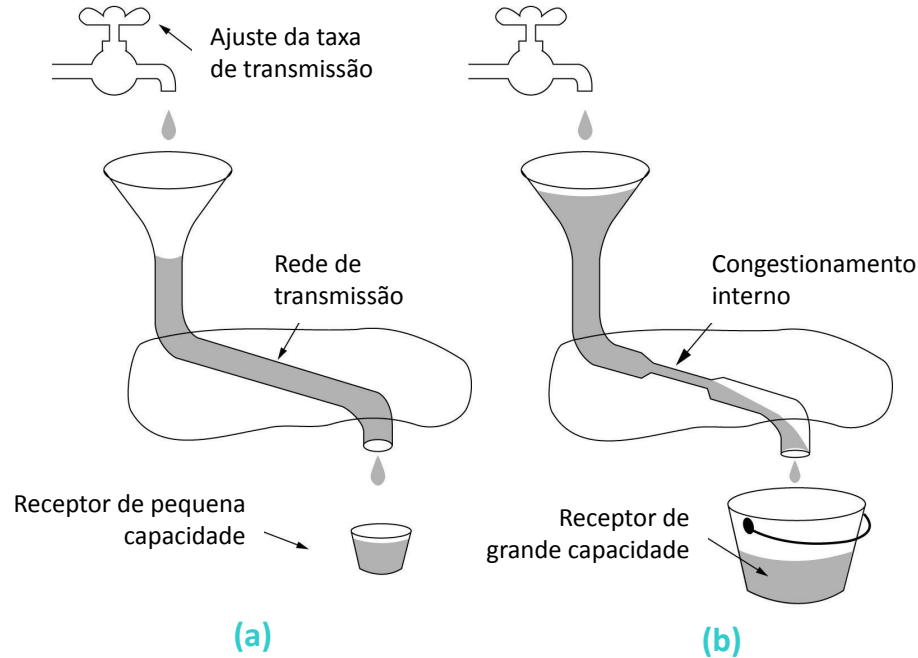
- Descreva o gerenciamento de conexões do TCP e mostre como os campos *SYN*, *ACK* e *FIN* funcionam no estabelecimento e término de conexões TCP

Exercício (7)

- Descreva o controle de fluxo do TCP e mostre como o campo *Window Size* funciona nesse caso

Exercício (8)

- Explique a figura abaixo no contexto dos controles de fluxo e de congestionamento do TCP

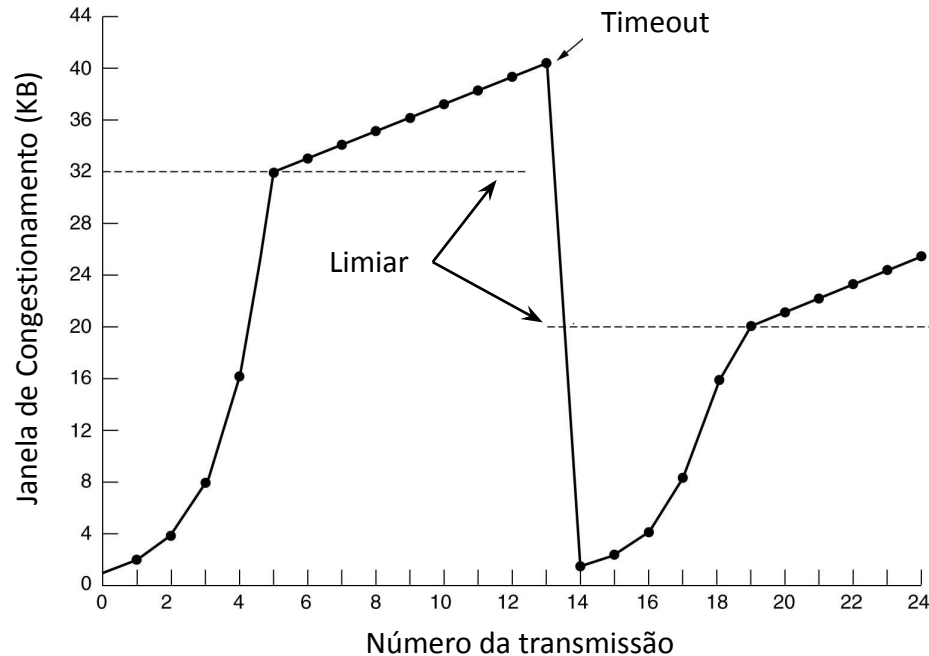


Exercício (9)

- Descreva o controle de congestionamento do TCP e mostre como funciona o algoritmo de iniciação lenta

Exercício (10)

- A figura abaixo mostra o tamanho da janela de congestionamento de um nó à medida que ele efetua transmissões em uma mesma conexão, explique a figura e os valores da janela de congestionamento



Exercício (11)

- Abra o Wireshark e faça a captura de pacotes. Em seguida, abra o www.google.com e faça algumas pesquisas. Volte para o Wireshark, utilize um dos dois filtros abaixo e explique os campos SEQ e ACK:
 - `(ipv6.addr == seu-ipv6 || ipv6.dst == seu-ipv6) && (tcp.dstport == 80 || tcp.srcport == 80)`
 - `(ip.addr == seu-ipv4 || ip.dst == seu-ipv4) && (tcp.dstport == 80 || tcp.srcport == 80)`